**Subject Code:** EP 206

**Course Title:** Microprocessor and Interfacing

**Contact Hours:** L: 3 T: 0 P: 2

**Examination Duration (Hrs.):** Theory: 3 Practical: 0

**Relative Weight:** CWS: 15 PRS:25 MTE: 20 ETE: 40 PRE: 0

**Credits:** 4

**Semester:** EVEN

**Subject Area:** DCC

**Pre-requisite:** NIL

**Objective:**

To familiarize the student with the concept of Microprocessors, memory organization, addressing modes and programming.



| S. No. | Contents | Contact Hours |
|---|---|---|
| 1. | **Basic Concepts of Microprocessors, Introduction to 8086 Microprocessor** | 2 |
| 2. | Internal architecture, **Concept of address, data and control buses** | 2 |
| 3. | **8086 hardware specifications: pin-outs and the pin-functions** | 2 |
| 4. | Real Mode Memory Addressing, Introduction to protected mode memory addressing | 2 |
| 5. | **Memory Address Space Organization.** | 2 |

| S. No. | Contents | Contact Hours |
|---|---|---|
| 6. | **Programming model of 8086-general purpose registers, special purpose registers and segment registers** | 2 |
| 7. | **Physical address generation**, data addressing modes, program memory addressing modes, stack memory addressing modes | 3 |
| 8. | data transfer instructions, arithmetic and logic instructions, flag control instructions, program control instructions | 2 |
| 9. | Input/Output instructions | 3 |
| 10. | Types of Interrupts, interrupt instructions, hardware interrupt interface, software interrupts, NMI interrupt | 2 |



| S. No. | Contents | Contact Hours |
|---|---|---|
| 11. | **Bus Cycle Timing Diagrams.** | 3 |
| 12. | **Minimum and Maximum mode** | 1 |
| 13. | Subroutines: Call and Return Functions. | 1 |
| 14. | Programmable Interrupt Controller - 8259 | 2 |
| 15. | Programmable Peripheral Interface (PPI) 8255 | 2 |

| S. No. | Contents | Contact Hours |
|---|---|---|
| 16. | Programmable Direct Memory Access (DMA) Controller - 8237/8257, | 2 |
| 17. | Programmable Interval Timer - 8253. | 2 |
| 18. | Introduction to PIC Microcontrollers,<br>PIC microcontroller overview and features, PIC 16F877: ALU, CPU registers, pin diagram | 2 |
| 19. | PIC reset actions, PIC oscillator connections, PIC memory organization, | 2 |
| 20. | PIC 16F877 instructions, Addressing modes, I/O ports. | 1 |
| 21. | Interfacing applications of Microcontroller-interfacing of 7 segment display, LCD interfacing, ADC and DAC interfacing. | 2 |



| S.No. | Name of Books/ Authors | Year of Publication/ Reprint |
|---|---|---|
| 1. | Y. Liu and G. A. Gibson, Microcomputer Systems: The 8086/8088 Family., Prentice Hall of India. | 2nd Ed |
| 2. | Douglas Hall, Microprocessors Interfacing, Tata McGraw Hill. | |
| 3. | Barry B. Brey, The Intel Microprocessors., Prentice Hall of India. | 7th Ed |
| 4. | Walter A. Treibel and Avtar Singh, The 8088 and 8086 Microprocessors, Prentice Hall of India. | |
| 5. | Rafiquzzaman, Microprocessors, Prentice Hall of India. | |
| 6. | A.K.Ray, K.M.Bhurchandi, Advanced Microprocessors and Peripherals, TMH. | Second Edition |
| 7. | Microcontroller and Embedded systems- M.A.Mazadi, J.G.Mazadi & R.D.McKinlay - Pearson PHI. | |
| 8. | Embedded Design with Microcontrollers by Martin Bates. | |

**Bit :** A digit of the binary number or code is called bit.

**Nibble :** The 4-bit (4-digit) binary number or code is called nibble.

**Byte :** The 8-bit (8-digit) binary number or code is called byte.

**Word :** The 16-bit (16-digit) binary number or code is called word.



**Double Word :** The 32-bit (32-digit) binary number or code is called double word.

**Multiple Word :** The 64, 128, ... bit /digit binary numbers or codes are called multiple words.

**Data :** The quantity (binary number/code) operated by an instruction of a program is called data. The size of data is specified as bit, byte, word, etc.

**Address :** Address is an identification number in binary for memory locations. The 8086 processor uses a 20-bit address for memory.

**Memory Word Size :** The memory word size or addressability is the size of binary information (or Addressability) that can be stored in a memory location. The memory word size for an 8086 processor-based system is 8-bit.

**Microprocessor :**

- The microprocessor is a program controlled semiconductor device(IC), which fetches (from memory), decodes and executes instructions.
- It is used as CPU (Central Processing Unit) in computers.
- The basic functional blocks of a microprocessor are ALU (Arithmetic Logic Unit), an array of registers and a control unit.
- The microprocessor is identified with the size of data, the ALU of the processor can work with at a time. The 8086 processor has a 16-bit ALU, hence it is called a 16-bit processor.
- The 80486 processor has a 32-bit ALU, hence it is called a 32-bit processor.



Bus : A bus is a group of conducting lines that carries data, address and control signals.

- Buses can be classified into Data bus, Address bus and Control bus.
- The group of conducting lines that carries data is called data bus.
- The group of conducting lines that carries address is called address bus.
- The group of conducting lines that carries control signals is called control bus.

**CPU Bus :** The group of conducting lines that are directly connected to the microprocessor is called CPU bus.

In a CPU bus, **the signals are multiplexed**, i.e., more than one signal is passed through the same line but at different timings.

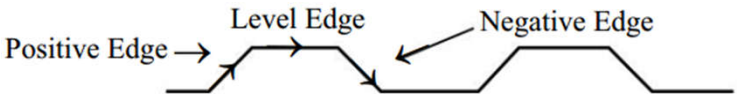
**System Bus :** The group of conducting lines that carries data, address and control signals in a microcomputer system is called System bus. **Multiplexing is not allowed in a system bus**



- an 8086 processor the address and data are sent through the same pins but at different timings.
- But when the system is formed, the multiplexed bus lines should be demultiplexed by using latches, ports, transceivers, etc.
- The demultiplexed bus lines are called system bus. In a system bus, separate conducting lines will be provided for each bit of data, address and control signals.

**Clock :**

- A clock is a square wave used to synchronize various devices in the microprocessor and in the system.
- Every microprocessor system requires a clock for its functioning.
- The time taken for the microprocessor and the system to execute an instruction or program are measured only in terms of the time period of its clock.






Tristate Logic :

Almost all the devices used in a microprocessor-based system use tristate logic.

In devices with tristate logic, three logic levels will be available:

High state, Low state and High impedance state.

**First Generation Microprocessors**

- The microprocessors introduced between 1971 and 1973 were the first generation processors.
- They were designed using PMOS technology. This technology provided low cost, slow speed and low output currents and was not compatible with TTL (Transistor Transistor Logic) levels.
- The first generation processors required a lot of additional support ICs to form a system, sometimes as high as 30 ICs. The 4-bit processors are provided with only 16 pins, but 8-bit and 16-bit processors are provided with 40 pins.



Due to limitations of pins, the signals are multiplexed. A list of first generation microprocessors are given below:

- INTEL 4004
- INTEL 4040
- FAIR CHILD PPS - 25
- NATIONAL IMP - 4
- ROCKWELL PPP - 4
- MICRO SYSTEMS INTL. MC - 1

} 4-bit processors

- INTEL 8008
- NATIONAL IMP - 8
- ROCKWELL PPS - 8
- AMI 7200
- MOSTEK 5065

} 8-bit processors

- NATIONAL IMP/16
- NATIONAL PACE

} 16-bit processors

Second Generation Microprocessors

- The second generation microprocessors appeared in 1973 and were manufactured using the NMOS technology.
- The NMOS technology offers faster speed and higher density than PMOS and it is TTL compatible.



- Some of the second generation processors are given below:

| | |
|---|---|
| INTEL 8080 | 8-bit processors |
| INTEL 8085 | |
| FAIRCHILD F - 8 | |
| MOTOROLA M6800 | |
| MOTOROLA M6809 | |
| NATIONAL CMP -8 | |
| RCA COSMAC | |
| MOS TECH. 6500 | |
| SIGNETICS 2650 | |
| ZILOG Z80 | |
| INTERSIL 6100 | 12-bit processors |
| TOSHIBA TLCS - 12 | |
| TI TMS 9900 | 16-bit processors |
| DEC - W.D. MCP - 1600 | |
| GENERAL INSTRUMENT CP 1600 | |
| DATA GENERAL μN601 | |

Characteristics of second generation microprocessors

- Larger chip size (170 × 200 mils). [1mil = $10^{-3}$inch]
- 40 pins.
- More numbers of on-chip decoded timing signals.
- The ability to address large memory spaces.
- The ability to address more IO ports.
- Faster operation.
- More powerful instruction set.
- A greater number of levels of subroutine nesting.
- Better interrupt handling capabilities.

## Third Generation Microprocessors

- After 1978, the third generation microprocessors were introduced.
- These are 16-bit processors and designed using HMOS (High density MOS) technology.
- Some of the third generation microprocessors are given below:

INTEL 8086 INTEL 80286 ZILOG Z8000
INTEL 8088 MOTOROLA 68000 NATIONAL NS 16016
INTEL 80186 MOTOROLA 68010 TEXAS INSTRUMENTS TMS 99000

The HMOS technology offers better Speed Power Product (SPP) and higher packing density than NMOS.
Speed Power Product = Speed × Power
= Nanosecond × Milliwatt
= Picojoules



Speed Power Product of HMOS is four times better than NMOS.

SPP of NMOS = 4 picojoules (pJ)

SPP of HMOS = 1 picojoules (pJ)

Circuit densities provided by HMOS are approximately twice those of NMOS.

Packing density of NMOS = 1852.5 gates/$mm^2$

Packing density of HMOS = 4128 gates/$mm^2$(1 mm = $10^{-6}$ meter)

**Characteristics of third generation microprocessors**

Provided with 40/48/64 pins.

High speed and very strong processing capability.

Easier to program.

Allow for dynamically relocatable programs.

Size of internal registers are 8/16/32 bits.

The processor has multiply/divide arithmetic hardware.

Physical memory space is from 1 to 16 Mega bytes.

The processor has segmented addresses and virtual memory features.

More powerful interrupt handling capabilities.

Flexible IO port addressing.

Different modes of operations (e.g., user and supervisor modes of M68000).



## Fourth Generation Microprocessors

- The fourth generation microprocessors were introduced in the year 1980.
- These generation processors are 32-bit processors and are fabricated using the low-power version of the HMOS technology called HCMOS.
- These 32-bit microprocessors have increased sophistications that compete strongly with mainframes. Some of the fourth generation microprocessors are given below:

INTEL 80386 MOTOROLA M68020 MOTOROLA MC88100

INTEL 80486 BELLMAC - 32

NATIONAL NS16032 MOTOROLA M68030

- Characteristics of fourth generation microprocessors

  Physical memory space of $2^{24}$ bytes = 16 Mb (Mega bytes).

  Virtual memory space of $2^{40}$ bytes = 1Tb (Tera bytes).

  Floating-point hardware is incorporated.

  Supports increased number of addressing modes.



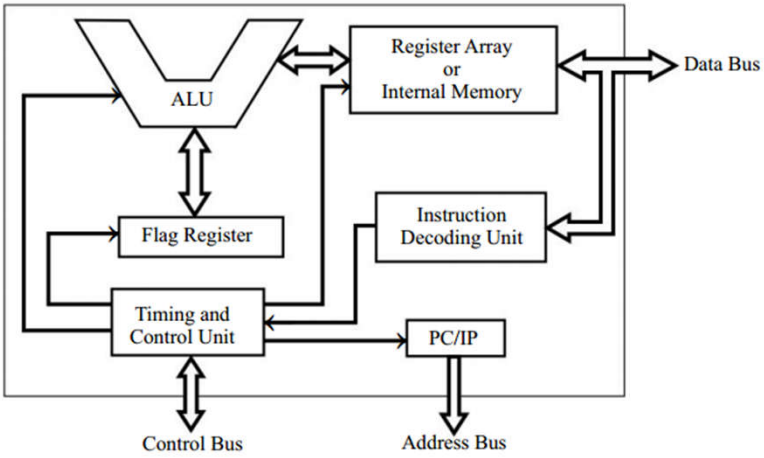
| MICROPROCESSOR | DATE OF INTRODUCTION | NUMBER OF TRANSISTORS | CLOCK SPEED |
|---|---|---|---|
| 4004 | 15th Nov, 1971 | 2,300 | 400 kHz |
| 8008 | Apr, 1972 | 3,500 | 500-800 kHz |
| 8080 | Apr, 1974 | 4,500 | 2 MHz |
| 8085 | Mar, 1976 | 6,500 | 5 MHz |
| 8086 | 8th Jun, 1978 | 29,000 | 5/8/10 MHz |
| 8088 | Jun, 1979 | 29,000 | 5/8 MHz |
| 80186 | 1982 | .... | 10/12 MHz |
| 80286 | Feb, 1982 | 134,000 | 6/10/12 MHz |
| INTEL386 DX | 17th Oct, 1985 | 275,000 | 16/20/25/33 MHz |
| INTEL386 SX | 16th Jun, 1988 | 275,000 | 16/20/25/33 MHz |
| INTEL386 SL | 15th Oct, 1990 | 855,000 | 20/25 MHz |
| INTEL486 DX | 10th Apr, 1989 | 1.2 million | 25/33/50 MHz |
| INTEL486 SX | 16th Sep, 1991 | 900,000 | 16/20/25 MHz |
| INTEL486 SX | 21st Sep, 1992 | 1.185 million | 33 MHz |
| INTEL486 SL | 4th Nov, 1992 | 1.4 million | 20/25/33 MHz |
| INTELDX 2 | 3rd Mar, 1992 | 1.2 million | 50/66 MHz |
| INTELDX 4 | 7th Mar, 1994 | 3.2 million | 75/100 MHz |
| Pentium | 22nd Mar, 1993 | 3.1 million | 60/66 MHz |
| Pentium | 7th Mar, 1994 | 3.2 million | 75/90/100/120 MHz |
| Pentium | Jun, 1995 | 3.3 million | 133/150/166/200 MHz |
| Pentium Pro | 1st Nov, 1995 | 5.5 million | 150/166/180/200 MHz |
| Pentium (MMX) | 8th Jan, 1997 | 4.5 million | 166/200/233 MHz |
| Mobile Pentium (MMX) | 9th Sep, 1997 | 4.5 million | 200/233/266/300 MHz |
| Pentium II | 7th May, 1997 | 7.5 million 450 MHz | 233/266/300/333/350/400/ |
| Mobile Pentium II | 2nd Apr, 1998 | 7.5 million | 233/266/300 MHz |
| Mobile Pentium II | 25th Jan, 1999 | 27.4 million | 333/366/400 MHz |
| Pentium II Xeon | 29th Jun, 1998 | 7.5 million | 400/450 MHz |
| Celeron | 15th Apr, 1998 | 7.5 million | 266/300 MHz |
| Celeron | 24th Aug, 1998 | 19 million | 333 MHZ to 2.7 GHz |
| Mobile Celeron | 25th Jan, 1999 | 18.9 million | 266 MHz to 2.4 GHz |
| Pentium III | 26th Feb, 1999 | 9.5 million | 450/500/550/600 MHZ |

| MICROPROCESSOR | DATE OF INTRODUCTION | NUMBER OF TRANSISTORS | CLOCK SPEED |
|---|---|---|---|
| Pentium III | 25th Oct, 1999 | 28 million | 500 MHz to 1 GHz |
| Pentium III Xeon | 17th Mar, 1999 | 9.5 million | 500/550 MHz |
| Pentium III Xeon | 25th Oct, 1999 | 28 million | 600 to 900 MHz |
| Mobile Pentium III | 25th Oct, 1999 | 28 million | 400 MHz to 1 GHz |
| Mobile Pentium III | 30th Jul, 2001 | 44 million | 1/1.06/1.13/1.2/1.33 GHz |
| Pentium 4 | 20th Nov, 2000 | 42 million | 1.4/1.5/1.6/1.7/1.8/1.9/2 GHz |
| Pentium 4 | 27th Aug, 2001 | 55 million | 2 to 2.8 GHz |
| Pentium 4 (HT Technology) | 14th Nov, 2002 | 55 million | 2.4 to 3.3 GHz |
| Mobile Pentium 4 | 4th Mar, 2002 | 55 million | 1.5 to 3.2 GHz |
| INTEL Xeon | 21st May, 2001 | 42 million | 1.4/1.5/1.7/2 GHz |
| INTEL Xeon | 9th Jan, 2002 | 52 million | 1.8/2/2.2/2.4/2.6/2.8 GHz |
| INTEL Xeon | 18th Nov, 2002 | 108 million | 1.4 to 3.2 GHz |
| INTEL Itanium | May, 2001 | 25 million | 733/800 MHz |
| INTEL Itanium 2 | 8th Jul, 2002 | 220 million | 900 MHz/1 GHz |
| INTEL Itanium 2 | 30th Jun, 2003 | 410 million | 1/1.4/1.5 GHz |
| INTEL Pentium-M | 12th Mar, 2003 | 77 million | 900 MHz to 1.7 GHz |

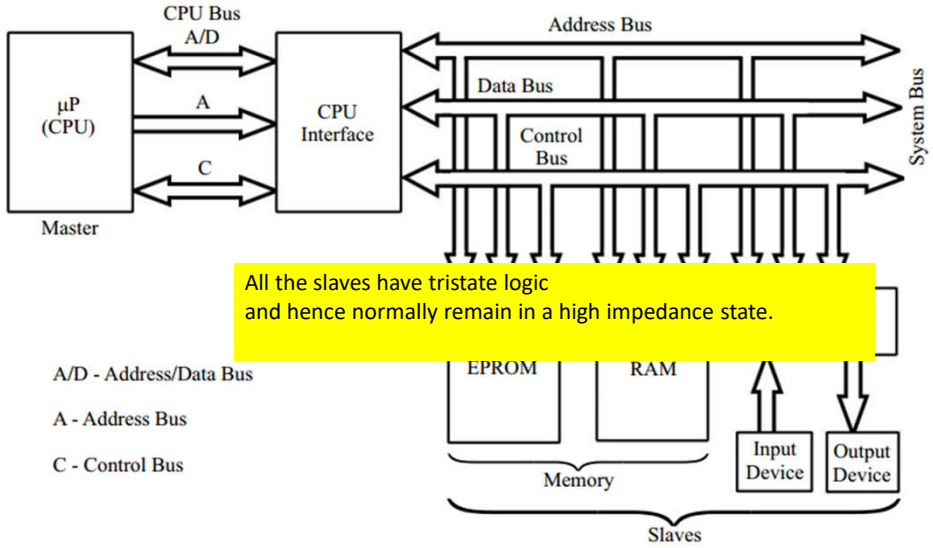
*Note : The date mentioned here is the date of introduction of the lowest clock version of the processor. For the date of introduction of higher clock version of a processor please refer to INTEL website www.intel.com.*



BASIC FUNCTIONAL BLOCKS OF A MICROPROCESSOR




Block diagram showing basic functional blocks of a microprocessor.




**Microprocessor-based system (organization of microcomputer).**



The commonly used EPROM and static RAM in microcomputers are given below:

| **EPROM** | **Static RAM** |
|---|---|
| INTEL 2708 (1 kb) | MOTOROLA 6208 (1 kb) |
| INTEL 2716 (2 kb) | MOTOROLA 6216 (2 kb) |
| INTEL 2732 (4 kb) | MOTOROLA 6232 (4 kb) |
| INTEL 2764 (8 kb) | MOTOROLA 6264 (8 kb) |

*Note : kb refer to kilo bytes.*

**CONCEPT OF MULTIPLEXING IN MICROPROCESSORS**

- Multiplexing is transferring different information at different well-defined times through the same lines.
- A group of such lines is called a multiplexed bus. The result of multiplexing is that fewer pins are required for microprocessors to communicate with the outside world.

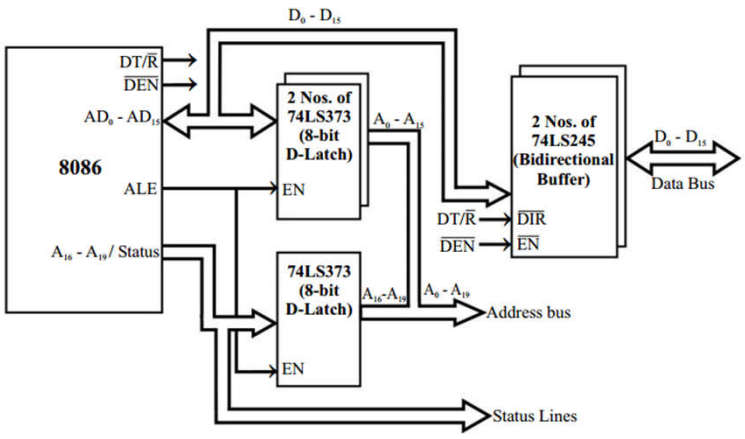


- Due to pin number limitations, most microprocessors cannot provide simultaneously similar lines (such as address, data, status signals, etc.).
- Hence multiplexing of one or more of these buses is performed. Most often data lines are multiplexed with some or all address lines to form an address/data bus (e.g., in 8086, the lower 16-address lines are multiplexed with data lines).
- The status signals emitted by the microprocessor are sometimes multiplexed either with the data lines (as done in INTEL 8080A) or with some of the address lines (as done in INTEL 8086).

Demultiplexing of Address/Data Lines in an 8086 Processor

- Signal called ALE (Address Latch Enable).
- The ALE is asserted high and then low by the processor at the beginning of every bus cycle.
- At the same time, the address is given out through $AD_0$ - $AD_{15}$ lines and $A_{16}$- $A_{19}$ status lines.
- Demultiplexing of address/data lines and address/status lines using 8- bit D-latch 74LS373 is shown in Fig






Demultiplexing of address and data lines in an 8086 processor.

- The ALE is connected to the Enable Pin (EN) of the external 8-bit latches.
- When ALE is asserted high and then low, the addresses are latched into the output lines of the latch.
- It holds the address until the next bus cycle. After latching the address, the AD0 - AD15 lines are free for data transfer and A 16 - A19 status lines are free for carrying status information.
- The first T-state of every bus cycle is used for address latching in 8086 and the remaining T states are used for reading or writing operation.

The data bus is provided with a bidirectional buffer in order to drive the data to a longer distance in the bus. The 8086 provides two control signals DT/$\overline{R}$ and $\overline{DEN}$ for controlling the data buffers. The DT/$\overline{R}$ is used to decide the direction of data flow and $\overline{DEN}$ is used to enable the data buffer.



Q: State the difference between CPU and ALU.

Q: Why is data bus bidirectional? Why is address bus unidirectional?

Q: What are the basic functional blocks of a microprocessor ?

Q: What is tristate logic? Why is it needed in a microprocessor system?

Q: What is HMOS and HCMOS?

Q: What is the function of a microprocessor in a system?

Q: What is multiplexing and what is its advantage?

# INTEL 8086 PINS, SIGNALS AND ARCHITECTURE



- INTEL 8086 is the first 16-bit processor released by INTEL in the year 1978.
- The 8086 was designed using the HMOS technology and it is now manufactured using HMOS III technology and contains approximately 29,000 transistors.
- The 8086 is packed in a 40-pin DIP and requires a single 5-V supply.

- **The 8086 does not have an internal clock circuit.**
- The 8086 requires **an external asymmetric clock source with 33% duty cycle.**
- The 8284 clock generator is used to generate the required clock for 8086.
- The maximum internal clock of 8086 is 5 MHz. The other versions of 8086 with different clock rates are 8086-1, 8086-2 and 8086-4 with maximum internal clock frequency of 10 MHz, 8 MHz and 4MHz respectively.



- The 8086 uses a 20-bit address to access memory and hence it can directly address up to one mega-byte ($2^{20}$ = 1 Mega) of memory space.
- One mega-byte (1Mb) of addressable memory space of 8086 are organized as **two memory banks of 512 kilo bytes each (512 kb + 512 kb = 1 Mb).**
- The memory banks are called even (or lower) bank and odd (or upper) bank.
- The address line A0 is used to select the even bank and the control signal $\overline{BHE}$ is used to select the odd bank.

For accessing IO-mapped devices, the 8086 uses a separate 16-bit address, and so the 8086 can generate 64 k($2^{16}$) IO addresses. The signal M/$\overline{IO}$ is used to differentiate the memory and IO addresses. For memory address, the signal M/$\overline{IO}$ is asserted **high** and for IO address the signal is M/$\overline{IO}$ asserted **low** by the processor.



- The 8086 can operate in two modes: minimum mode and maximum mode.
- The mode is decided by a signal at MN/$\overline{MX}$ pin.
- When the MN/$\overline{MX}$ is tied high, it works in minimum mode and the system is called a uniprocessor system.
- When MN/$\overline{MX}$ is tied low, it works in maximum mode and the system is called a multiprocessor system.

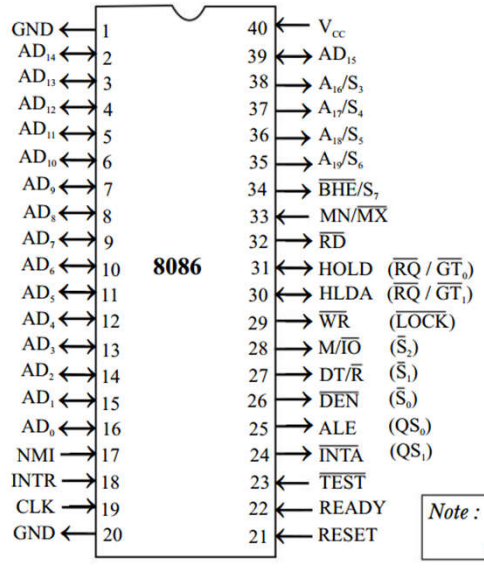
- Usually the pin $\mathrm{MN}/\overline{MX}$ is permanently tied to low or high so that the 8086 system can work in any one of the two modes.
- The 8086 can work with the 8087 coprocessor in maximum mode. (*In this mode an external bus controller 8288 is required to generate bus control signals.)*



- ❑ The 8086 has two families of processors.
- ❑ They are 8086 and 8088.
- ❑ The 8088 uses 8-bit data bus externally but the 8086 uses 16-bit data bus externally.
- ❑ The 8086 accesses memory in words but the 8088 accesses memory in bytes.
- ❑ IBM designed its first Personal Computer (PC) using an INTEL 8088 microprocessor as the CPU.

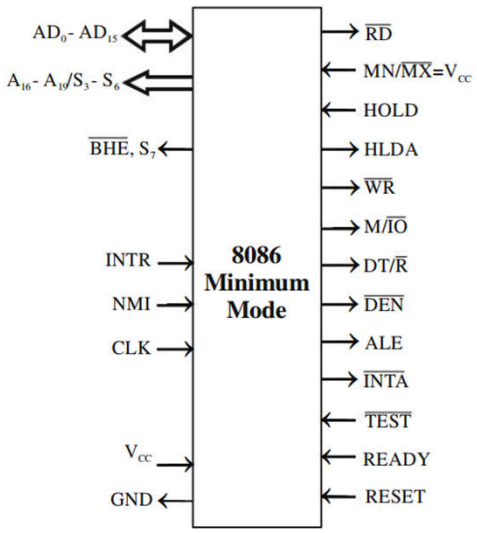
- The 8086 is a 40-pin IC and all the 8086 pins are TTL compatible.
- **The signal assigned to pins 24 to 31 will be different for minimum and maximum mode of operation.**
- The signal assigned to all other pins are common for minimum and maximum mode of operation.



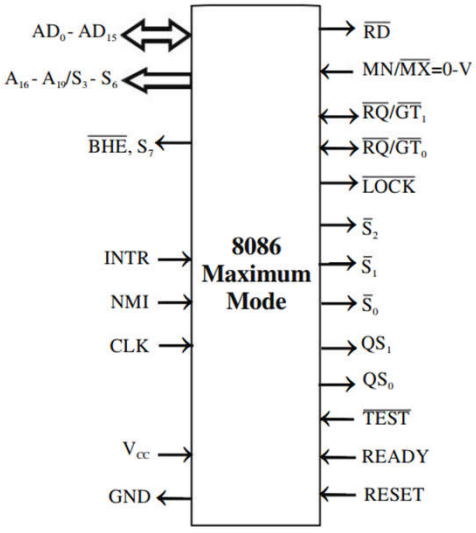



8086 pin assignments.

- **The 8086 is a 40-pin IC and all the 8086 pins are TTL compatible.**
- **The signal assigned to pins 24 to 31 will be different for minimum and maximum mode of operation.**
- **The signal assigned to all other pins are common for minimum and maximum mode of operation.**




8086-Minimum mode



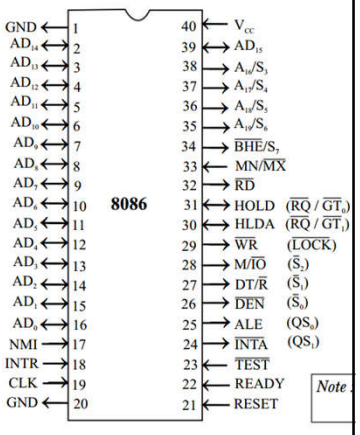

8086-Maximum mode

**8086 pin and sign**



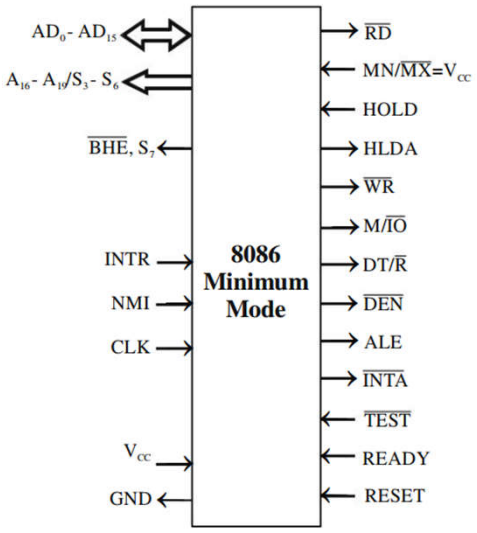
**COMMON SIGNALS**

| Name | Description/Function | Type |
|---|---|---|
| $AD_{15}$ - $AD_0$ | Address/Data | Bidirectional, Tristate |
| $A_{19}/S_6$ - $A_{16}/S_3$ | Address/Status | Output, Tristate |
| $\overline{BHE}/S_7$ | Bus high enable/Status | Output, Tristate |
| MN/$\overline{MX}$ | Minimum/Maximum mode control | Input |
| $\overline{RD}$ | Read control | Output, Tristate |
| $\overline{TEST}$ | Wait on test control | Input |
| READY | Wait state control | Input |
| RESET | System reset | Input |
| NMI | Non-maskable interrupt request | Input |
| INTR | Interrupt request | Input |
| CLK | System clock | Input |
| $V_{cc}$ | +5-V | Power supply input |
| GND | Ground | Power supply ground |



**MINIMUM MODE SIGNALS** [MN/$\overline{MX}$ = $V_{cc}$ (Logic high)]

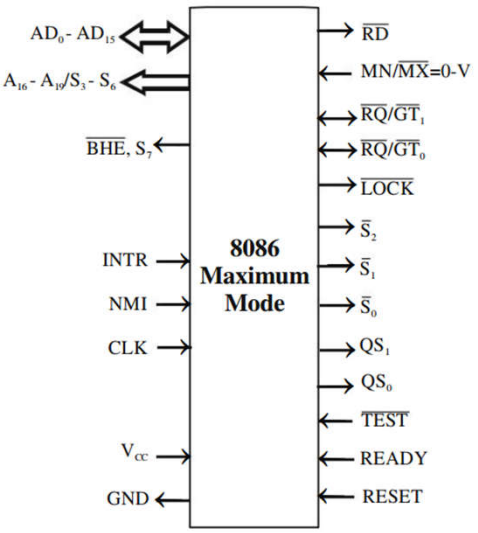
| Name | Description/Function | Type |
|---|---|---|
| HOLD | Hold request | Input |
| HLDA | Hold acknowledge | Output |
| $\overline{WR}$ | Write control | Output, Tristate |
| M/$\overline{IO}$ | Memory/IO control | Output, Tristate |
| DT/$\overline{R}$ | Data transmit/Receive | Output, Tristate |
| $\overline{DEN}$ | Data enable | Output, Tristate |
| ALE | Address latch enable | Output |
| $\overline{INTA}$ | Interrupt acknowledge | Output |






**MAXIMUM MODE SIGNALS** [MN/$\overline{MX}$ = Ground(Logic low)]

| Name | Description/Function | Type |
|---|---|---|
| $\overline{RQ}/\overline{GT_1}$, $\overline{RQ}/\overline{GT_0}$ | Request/Grant bus access control | Bidirectional |
| $\overline{LOCK}$ | Bus priority lock control | Output, Tristate |
| $\overline{S_2}$, $\overline{S_1}$, $\overline{S_0}$ | Bus cycle status | Output, Tristate |
| $QS_1$, $QS_0$ | Instruction queue status | Output |



**Common Signals**

- The common signals for minimum and maximum mode are listed in Table.
- The lower sixteen lines of the address are multiplexed with data and the upper four lines of the address are multiplexed with status signals.
- During the first clock period of a bus cycle the entire 20-bit address is available on these lines. During all other clock periods of a bus cycle, the data and status signals will be available on these lines.



- The status signals on S3 and S4 specify the segment register used for calculating physical address.
- The output on the status lines S3 and S4 when the processor is accessing various segments are listed in Table

STATUS SIGNAL DURING MEMORY SEGMENT ACCESS

| Status signal | | Segment register |
|---|---|---|
| $S_4$ | $S_3$ | |
| 0 | 0 | Extra segment |
| 0 | 1 | Stack segment |
| 1 | 0 | Code or no segment |
| 1 | 1 | Data segment |

- The status lines $S_3$ and $S_4$ can be used to expand the memory up to 4 Mb.
- The status line $S_5$ indicates the status of the 8086 interrupt enable flag.
- A low on the line $S_6$ indicates that the 8086 is on the bus (i.e., it indicates that 8086 is the bus master) and during hold acknowledge, this pin is driven to high impedance state.
- The output signal $\overline{BHE}$ on the first T-state of a bus cycle is maintained as status signal S7 during all other T states of the bus cycles.



- The 8086 outputs a low on the $\overline{BHE}$ pin during read, write and interrupt acknowledge cycles when the data is to be transferred to the high order data bus.
- The $\overline{BHE}$ can be used in conjunction with address bit A0(AD0) to select memory banks.

When the processor reads from the memory or an IO location it asserts $\overline{RD}$ as **low**. The $\overline{TEST}$ input is tested by the WAIT instruction. The 8086 will enter a wait state after execution of the WAIT instruction, and it will resume execution only when $\overline{TEST}$ is made **low** by an external hardware. This is used to synchronize an external activity to the processor's internal operation. $\overline{TEST}$ input is synchronized internally during each clock cycle on the leading edge of the clock signal.



- INTR is the maskable interrupt and INTR must be held high until it is recognized to generate an interrupt signal.
- NMI is the non-maskable interrupt input activated by a leading edge signal.

- RESET is the system reset input signal.
- For power-ON reset, it is held high for 50 microseconds.
- For reset while working, it is held high for at least four clock cycles.
- When the processor is reset, the DS, SS, ES, IP and flag register are cleared, Code Segment (CS) register is initialized to $FFFF_H$ and queue is emptied.
- After reset, the processor will start fetching instructions from the 20-bit physical address $FFFF0_H$.



- READY is an input signal to the processor, used by the memory or IO devices to get extra time for data transfer or to introduce wait states in the bus cycles.
- Normally READY is tied high.
- If the READY is tied low, the 8086 introduces wait states after second T-state of a bus cycle and it will complete the bus cycle only when READY is made high again.

- CLK input is the clock signal that provides the basic timing for the 8086 and bus controller.
- The 8086 does not have an on-chip clock generation circuit. Hence the 8284 clock generator chip is used to generate the required clock.
- A quartz crystal whose frequency is thrice that of internal clock of an 8086 must be connected to the 8284.



- The 8284 generates the clock at crystal frequency.
- The 8284 divides the generated clock by three and modifies the duty cycle to 33% and output on CLK pin of 8284.
- This CLK output of 8284 must be connected to the 8086 CLK pin.
- The 8284 also provides the RESET and READY signals to the 8086.

## Minimum Mode Signals

For minimum mode of operation the $\mathrm{MN}/\overline{MX}$ pin is tied to Vcc (logic high). In minimum mode, the 8086 itself generates all bus control signals.

**DT/$\overline{\mathbf{R}}$** - (*Data Transmit/Receive*). It is an output signal from the processor to control the direction of data flow through the data transceivers.

**$\overline{\mathbf{DEN}}$** - (*Data Enable*) - It is an output signal from the processor used as output enable for the data transceivers.

**ALE** - (*Address Latch Enable*) - It is used to demultiplex the address and data lines using external latches.

**M/$\overline{\mathbf{IO}}$** - It is used to differentiate memory access and IO access. For IN and OUT instructions it is **low**. For memory reference instructions, it is **high.**

**$\overline{\mathbf{WR}}$** - It is a write control signal and it is asserted **low** whenever the processor writes data to memory or IO port.



**$\overline{\mathbf{INTA}}$ - (Interrupt Acknowledge) -** The 8086 outputs low on this line to acknowledge when the interrupt request is accepted by the processor.

**HOLD -** It is an input signal to the processor from other bus masters as a request to grant the

control of the bus. It is usually used by DMA controller to get the control of bus.

**HLDA - (Hold Acknowledge)** –

- It is an acknowledge signal by the processor to the master requesting the control of the bus through HOLD.
- The acknowledge is asserted high when the processor accepts the HOLD.

*[On accepting the hold, the processor drives all the tristate pins to high impedance state and sends an acknowledge to the device which requested HOLD.*

*On receiving the acknowledge, the other master will take control of the bus.]*



## Maximum Mode signals

For maximum mode of operation the MN/$\overline{MX}$ pin is tied to GND (logic zero).

$\overline{S_0}$ , $\overline{S_1}$, $\overline{S_2}$ These are status signals and they are used by the 8288 bus controller to generate the bus timing and control signals.

STATUS SIGNALS DURING VARIOUS MACHINE CYCLES

| Status signal | | | Machine cycle |
|---|---|---|---|
| $\overline{S}_2$ | $\overline{S}_1$ | $\overline{S}_0$ | |
| 0 | 0 | 0 | Interrupt acknowledge |
| 0 | 0 | 1 | Read IO port |
| 0 | 1 | 0 | Write IO port |
| 0 | 1 | 1 | Halt |
| 1 | 0 | 0 | Code access |
| 1 | 0 | 1 | Read memory |
| 1 | 1 | 0 | Write memory |
| 1 | 1 | 1 | Passive/Inactive |

$\overline{RQ}/\overline{GT}_0$, $\overline{RQ}/\overline{GT}_1$ - (Bus Request/Bus Grant) These requests are used by the other local bus masters to force the processor to release the local bus at the end of the processor's current bus cycle. These pins are bidirectional. The request on $GT_0$ will have higher priority than $GT_1$.



The bus request to 8086 work as follows:

1. When a local bus master requires system bus control, it sends a low pulse to the 8086.

2. At the end of the current bus cycle, the processor (8086) drives its pins to high impedance state and sends an acknowledge as a low pulse on the same pin to the device which requested the bus control.

3. On receiving the acknowledge the local master will take control of the system bus. After completing its work, at the end, the local bus master sends a low signal on the same pin to the 8086 to inform the end of control.

Now 8086 regains the control of the bus.



**$\overline{\text{LOCK}}$** - It is an output signal, activated by the LOCK prefix instruction and remains active until the completion of the instruction prefixed by LOCK. The 8086 outputs **low** on the **$\overline{\text{LOCK}}$** pin while executing an instruction prefixed by LOCK to prevent other bus masters from gaining control of the system bus.

**QS1, QS0 - (Queue Status) -** The processor provides the status of queue on these lines.

- The queue status can be used by the external device to track the internal status of the queue in the 8086.
- The $QS_0$ and $QS_1$ are valid during the clock period following any queue operation.



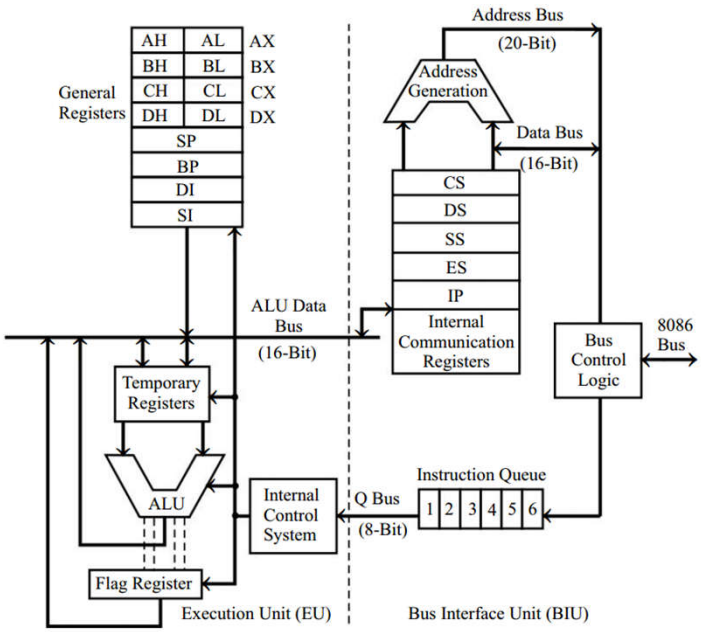
**QUEUE STATUS**

| Queue status | | Queue operation |
|---|---|---|
| $QS_1$ | $QS_0$ | |
| 0 | 0 | No operation |
| 0 | 1 | First byte of an opcode from queue |
| 1 | 0 | Empty the queue |
| 1 | 1 | Subsequent byte from queue |

# ARCHITECTURE OF INTEL 8086



- The 8086 has a pipelined architecture.
- In pipelined architecture, the processor will have a number of functional units and the execution time of each functional unit overlaps.
- Each functional unit works independently most of the time.
- The simplified block diagram of the internal architecture of an 8086 is shown in Fig.






**Internal architecture of 8086**



The architecture of the 8086 can be internally divided into two separate functional units :

- Bus Interface Unit (BIU)
- Execution Unit (EU)

- The BIU fetches instructions, reads data from memory and IO ports, writes data to memory and IO ports.

The BIU contains

- ✓ segment registers
- ✓ instruction pointer
- ✓ instruction queue
- ✓ address generation unit
- ✓ bus control unit



- The EU executes instructions that have already been fetched by the BIU.
- The BIU and EU function independently.
- The instruction queue is a FIFO (First-In-First-Out) group of registers.
- The size of queue is 6 bytes.
- The BIU fetches instruction code from the memory and stores it in the queue.
- The EU fetches instruction codes from the queue.

- The BIU has four numbers of 16-bit segment registers.
- They are

a. Code Segment (CS) register,
b. Data Segment (DS) register,
c. Stack Segment (SS) register
d. Extra Segment (ES) register.

- The 8086 memory space can be divided into segments of 64 kilo bytes (64 kb).



- The 8086 memory space can be divided into segments of 64 kilo bytes (64 kb).
- The 4 segment registers are used to hold four segment base addresses.
- **Hence 8086 can directly address 4 segments of 64 kb at any time instant (4 × 64 = 256 kb within 1 Mb memory space).**
- This feature of 8086 allows the system designer to allocate separate areas for storing program codes and data.

- The contents of segment registers are programmable.
- **Hence, the processor can access the code and data in any part of the memory by changing the contents of the segment registers.**
- The memory segment can be continuous, partially overlapped, fully overlapped, or disjoint.

*Note :*
- *Since segment registers are programmable it is possible to design multitasking and multiuser system using 8086.*
- *The program code and data for each task/user can be stored in separate segments.*
- *The program execution can be switched from one task/user to another by changing the content of the segment registers.*



- The dedicated address generation unit generates the 20-bit physical address from the segment base and an offset or effective address.
- The segment base address is logically shifted left four times and added to the offset

*[Logically shifting left four times is equal to multiplying by $16_{10}$.]*

- The address for fetching instruction codes is generated by logically shifting the content of the CS to the left four times and then adding it to the content of the IP (Instruction Pointer).
- The IP holds the offset address of the program codes.
- The content of IP gets incremented by two after every bus cycle *[In one bus cycle, the processor fetches two bytes of the instruction code.]*



- The data address is computed by using the content of DS or ES as base address and an offset or effective address specified by the instruction.
- The stack address is computed by using the content of the SS as base address and the content of the SP (Stack Pointer) as the offset address or effective address.

- The **bus control logic of the BIU** generates all the bus control signals such as read and write signals for memory and IO.
- The EU consists of ALU, flag register and general purpose registers.
- The EU decodes and executes the instructions.
- A decoder in the EU control system translates the instructions.



- The EU has a 16-bit ALU to perform arithmetic and logical operations.
- The EU has eight numbers of 16-bit general purpose registers.
- They are **AX, BX, CX, DX, SP, BP, SI and DI.**

## SPECIAL FUNCTIONS OF 8086 REGISTERS

| Register | Name of the register | Special function |
|---|---|---|
| AX | 16-bit Accumulator | Stores the 16-bit result of certain arithmetic and logical operations. |
| AL | 8-bit Accumulator | Stores the 8-bit result of certain arithmetic and logical operations. |
| BX | Base register | Used to hold the base value in base addressing mode to access memory data. |
| CX | Count register | Used to hold the count value in SHIFT, ROTATE and LOOP instructions. |
| DX | Data register | Used to hold data for multiplication and division operations. |
| SP | Stack pointer | Used to hold the offset address of top of stack memory. |
| BP | Base pointer | Used to hold the base value in base addressing using stack segment register to access data from stack memory. |
| SI | Source index | Used to hold the index value of source operand (data) for string instructions. |
| DI | Destination index | Used to hold the index value of destination operand (data) for string instructions. |



*Some of the 16-bit registers can also be used as two numbers of 8-bit registers as given below:*

AX - can be used as AH and AL ;

BX - can be used as BH and BL ;

CX - can be used as CH and CL

DX - can be used as DH and DL

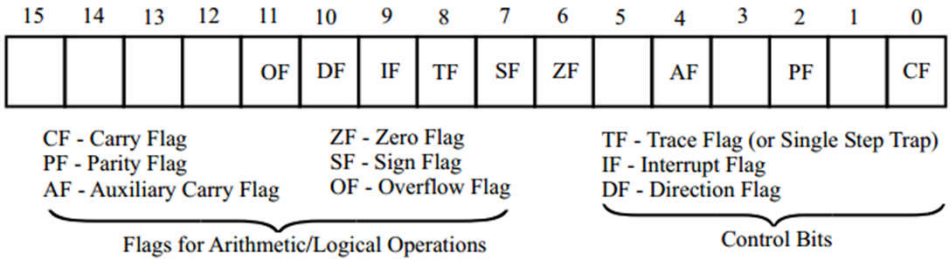
The general purpose registers can be used for data storage, when they are not involved in special functions assigned to them.

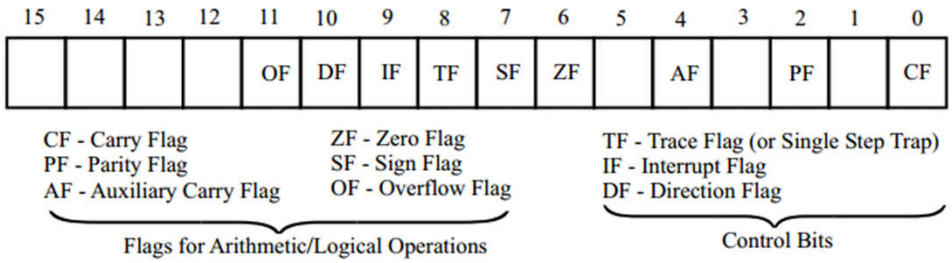
## 8086 Flag Register

- The size of an 8086 flag register is 16-bit and in this **nine bits** are defined as flags.
- The **six flags** are used to indicate **the status of the result of the arithmetic or logical operations.**
- **Three flags** are used to control the **processor operation** and so they are also called **control bits**.



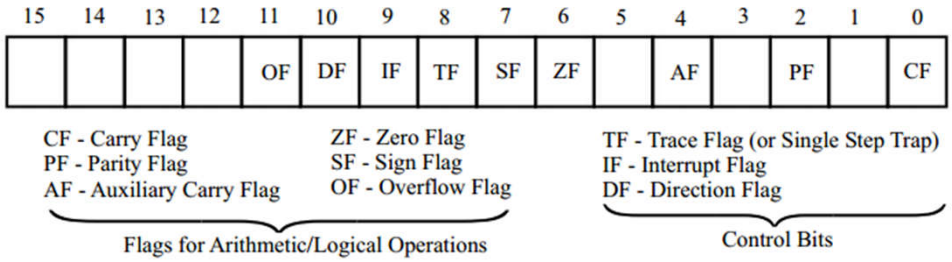
## Bit positions of various flags in the flag register of 8086



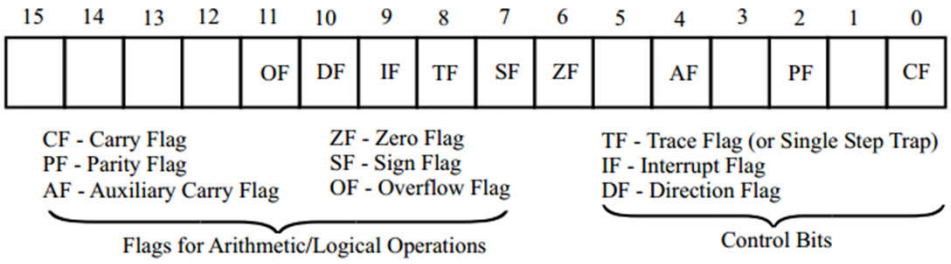



- Carry Flag (CF) is set if there is a carry from addition or borrow from subtraction.
- Auxiliary carry Flag (AF) is set if there is a carry from low nibble to high nibble of the low order 8-bit of a 16-bit number.



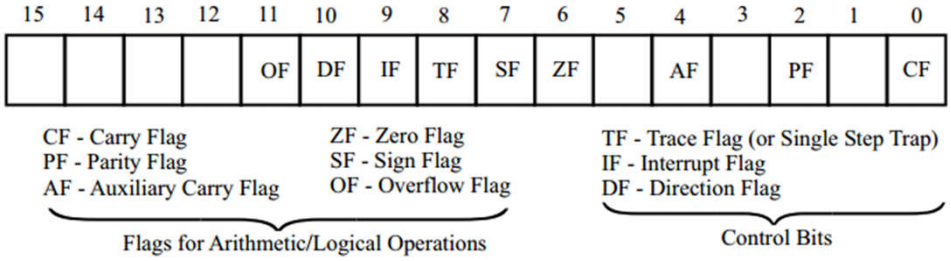



- Overflow Flag (OF) is set to one if there is an arithmetic overflow, that is, if the size of the result exceeds the capacity of the destination location.
- Sign Flag (SF) is set to one if the most significant bit of the result is one and SF is cleared to zero for non-negative result.



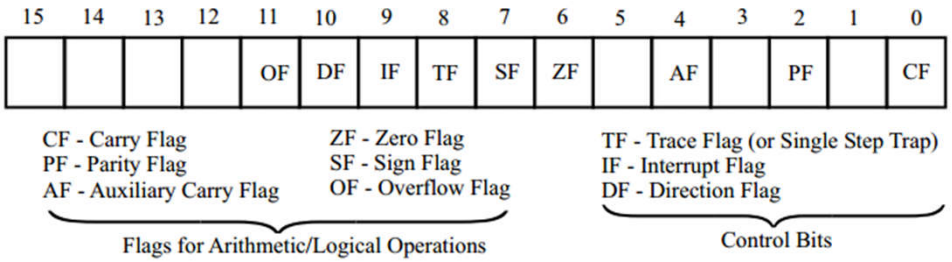

- Parity Flag (PF) is set to one if the result has even parity and PF is cleared to zero for odd parity of the result.
- Zero Flag (ZF) is set to one if the result is zero and ZF is cleared to zero for nonzero result.






The three control bits in the flag register can be set or reset by the programmer.

- The Direction Flag (DF) is set to one for autodecrement and DF is reset to zero for autoincrement of SI and DI registers during string data accessing.
- Setting Interrupt Flag (IF) to one causes the 8086 to recognize external maskable interrupts and clearing IF to zero disables the interrupts.




- Setting Trace Flag (TF) to one places the 8086 in the single-step mode.

In this mode, the 8086 generates an internal interrupt after execution of each instruction.

The single stepping is used for debugging a program.



## INSTRUCTION AND DATA FLOW IN 8086

- The 8086 microprocessor allows the user to define different memory areas for storing programs and data.
- The program memory can be accessed using CS-register, and the data memory can be accessed using DS, ES and SS registers.



- The program instructions are stored in program memory which is an external device.
- To execute a program in 8086, the base address and offset address of the first instruction of the program should be loaded in CS-register and IP, respectively.
- The 8086 computes the 20-bit physical address of the program instruction by multiplying the content of CS-register by $16_{10}$ and adding it to the content of IP.

- The 20-bit physical address is given out on the address bus.
- Then $\overline{RD}$ is asserted low. Also other control signals necessary for program memory read operation are asserted.
- The IP is incremented by two to point next instruction or next word of the same instruction.



- The address and control signals enable the memory to output one word (two bytes) of program memory on the data bus.
- After a predefined time, the $\overline{RD}$ is asserted high and at this instant the content of data bus is latched into two empty locations of instruction queue.
- Then BIU starts fetching the next word of the program code as explained above.

- The BIU keeps on fetching the program codes, word by word from consecutive memory locations whenever two locations of queue are empty.
- When a branch instruction is encountered, the queue is emptied and then filled with program codes from the new address loaded in CS and IP by the branch instruction.



- The EU reads the program instructions from queue, decodes and executes them one by one.
- If the execution of an instruction requires data from memory (or to store data in memory) then BIU is interrupted to read (or write) data in memory.

- When BIU is interrupted, it completes the fetching of current instruction word and then starts reading/writing the data by generating a 20-bit data memory address.
- The 20-bit data memory address is obtained by multiplying the content of segment base register specified by the instruction by $16_{10}$ and adding it to an effective or offset address specified by the instruction.



**EVEN AND ODD MEMORY BANKS**

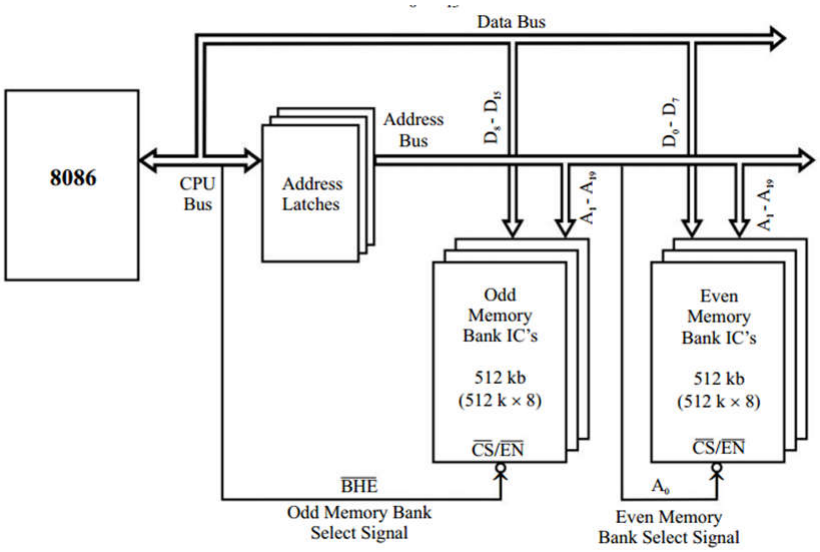
**EVEN AND ODD MEMORY BANKS**

- The 8086 microprocessor uses a 20-bit address to access memory.
- With 20-bit address the processor can generate $2^{20}$ = 1 Mega address.
- The basic memory word size of the memories used in the 8086 system is 8-bit or 1-byte (i.e., in one memory location an 8-bit binary information can be stored).
- Hence, the physical memory space of the 8086 is 1Mb (1 Mega-byte).



- For the programmer, the 8086 memory address space is a sequence of one mega-byte in which one location stores an 8-bit binary code/data and two consecutive locations store 16-bit binary code/data.
- But physically (i.e., in the hardware), the 1Mb memory space is divided into two banks of 512kb (512kb + 512kb = 1Mb).
- The two memory banks are called Even (or Lower) bank and Odd (or Upper) bank.

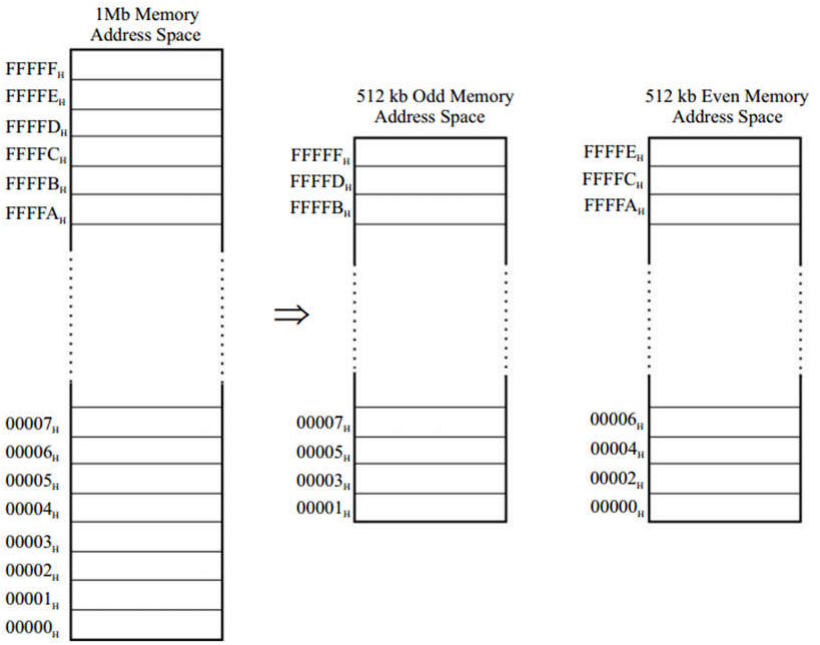
- The organization of even and odd memory banks in the 8086-based system is shown in Fig.




03-03-2023 16:40:13

*Figure Organization of even and odd memory banks in 8086-based system*

103




03-03-2023 16:40:13

*Figure Organization of even and odd memory banks in 8086-based system*

104

- The 8086-based system will have two sets of memory IC's. One set for even bank and another set for odd bank.
- The data lines D0-D7 are connected to even bank and the data lines D8-D15 are connected to odd bank.



- The even memory bank is selected by the address line A0 and the odd memory bank is selected by the control signal $\overline{\mathrm{BHE}}$.
- The memory banks are selected when these signals are low(active low).
- Any memory location in the memory bank is selected by the address line A1 to A19.

- The organization of memory into two banks and providing bank select signals allows the programmer to read/write the byte (8-bit) operand in any memory address through 16-bit data bus.
- It also allows the programmer to read/write the word (16-bit) operand starting from even address or odd address.



The memory access for byte and word operand from the even and odd bank by the 8086 processor will be as follows:

**Case i : Byte access from even bank**

For read/write operation of a byte in even memory address, A0 is asserted low and $\overline{\text{BHE}}$ is asserted high **(i.e., A0 = 0 and $\overline{\text{BHE}}$ = 1)**.

- Now the even bank alone is enabled and the data transfer take place through D0-D7 data lines.

**Case ii : Byte access from odd bank**

- For read/write operation of a byte in odd memory address, A0 is asserted high and $\overline{\mathrm{BHE}}$ is asserted low **(i.e., A0 = 1 and $\overline{\boldsymbol{BHE}}$ = 0)**.
- Now odd bank alone is enabled and the data transfer take place through D8-D15 data lines.



**Case iii : Word access from even boundary**

- For read/write operation of a word (16-bit) in even boundary (i.e., low byte in even address and high byte in next address (odd address)), both A0 and $\overline{\mathrm{BHE}}$ are asserted low **(i.e., A0 = 0 and $\overline{\boldsymbol{BHE}}$ = 0).**
- Now both the memory banks are enabled simultaneously and the processor read /write the 16-bit operand in one bus cycle through D0-D15 data lines.

**Case iv : Word access from odd boundary**

- For read/write operation of a word (16-bit) in odd boundary (i.e., low byte in odd address and high byte in next address (even address)), the processor executes two bus cycles to read/write the word (16-bit) operand.



- In the first bus cycle A0 is asserted high and BHE is asserted low (i.e., A0 = 1 and $\overline{\mathrm{BHE}}$ = 0).
- Now the odd bank alone is enabled and the low byte of 16-bit operand is read/write through D8-D15 data lines. In the second bus cycle A0 is asserted low and BHE is asserted high (i.e., A0 = 0 and $\overline{\mathrm{BHE}}$ = 1).
- Now the even bank alone is enabled and the high byte of 16-bit operand is read/write through D0-D7 data lines.

*STATUS OF $A_0$ AND $\overline{BHE}$ FOR BYTE AND WORD DURING MEMORY ACCESS*

| Memory bank | Operand type | Status of | | Data lines used for memory access | No. of bus cycle |
|---|---|---|---|---|---|
| | | $A_0$ | $\overline{BHE}$ | | |
| Even | Byte | 0 | 1 | $D_0$ - $D_7$ | One |
| Odd | Byte | 1 | 0 | $D_8$ - $D_{15}$ | One |
| Even | Word | 0 | 0 | $D_0$ - $D_{15}$ | One |
| Odd | Word | 1 | 0 | $D_8$ - $D_{15}$ | First cycle |
| | | 0 | 1 | $D_0$ - $D_7$ | Second cycle |

*Note :* 1. *The processor may access the low byte operand via the upper data lines and high byte operand via the lower data lines, but while placing the operand in the registers it places in the appropriate locations.*

2. *When word operand is accessed from odd boundary the instruction execution will take extra time due to two bus cycle memory access.*



## BUS CYCLES AND TIMING DIAGRAM

- The 8086 processor has two functional units called

  Bus Interface Unit (BIU)

  Execution Unit (EU)

- ❑ Most of the time, each unit works independently.
- ❑ The BIU fetches instruction codes from memory and data from memory and IO devices.
- ❑ The EU takes care of executing the instructions prefetched by BIU.



- The BIU initiates all external operations which are also called bus activity.
- The external bus activities are repetitions of certain basic operations.
- The basic operations performed by the CPU bus are called bus cycles.
- Depending on the activities of 8086, the bus cycles can be classified as follows:

1) Memory read cycle (Four T states)
2) Memory write cycle (Four T states)
3) IO read cycle (Four T states)
4) IO write cycle (Four T states)
5) Interrupt acknowledge cycle (Eight T states)

- The processor takes a definite time to perform a bus cycle.
- The time taken to perform a bus cycle are specified in terms of T states.
- In an 8086 processor the time duration of one T-state is equal to one time period of the internal clock of the processor.
- The T-state starts in the middle of falling edge of the clock signal as shown in Fig.

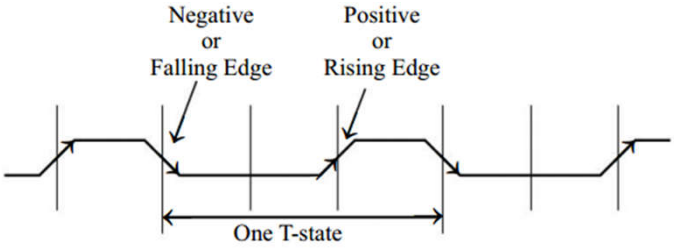



*Fig. : Clock signal and one T-state of 8086*



- The normal time taken by 8086 to perform read/write cycle is four T states.
- The processor also has facility to extend the timing of bus cycles by introducing extra T states called wait states using READY control signal.
- If READY is permanently tied high then the bus cycles are executed in normal timing

- The memory or IO access time allowed by the 8086 processor with 5 MHz clock is 400 ns.
- If the memory or IO devices used in the system has access time more than 400 ns then wait states have to be introduced in the bus cycles, between the second and third T states (T2 and T3) to extend the timing of the bus cycle.



- In order to introduce wait states the READY is made low by an external hardware in the beginning of second T-state and then made high after required time delay.
- The processor samples READY signal at the end of second T-state of every bus cycle.
- If READY is low at this time then the processor introduces one wait state.

- Again it samples READY signal at the end of wait state and if READY is still low then it introduces another wait state.
- This process is continued until READY is high.
- Once READY is made high the processor will resume the bus activity and completes the bus cycle.



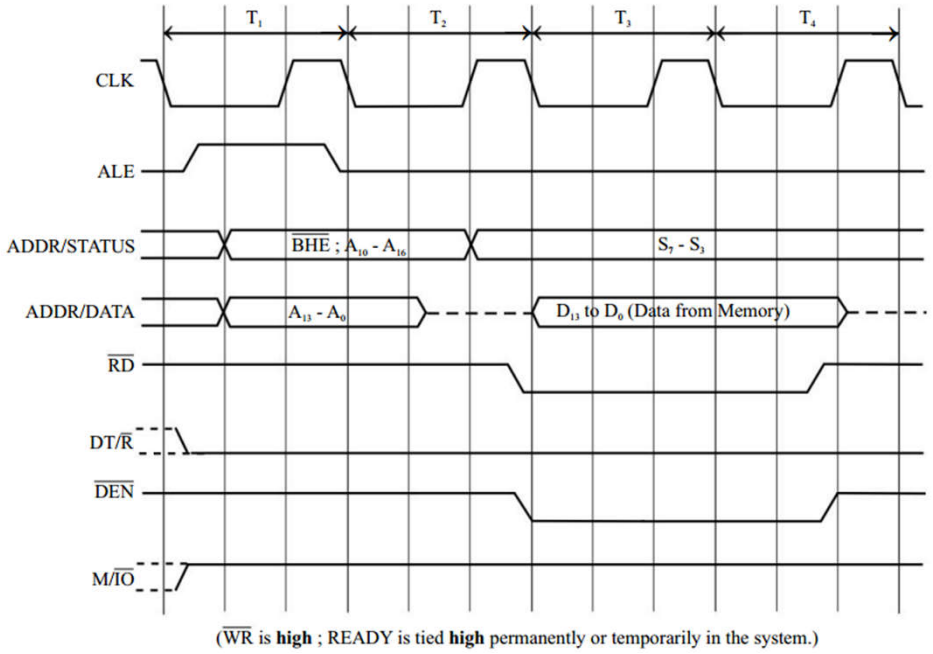
**Timing Diagram**

- The timing diagram provides information about the various conditions (high state or low state or high impedance state) of the signals while a bus cycle is executed.
- The timing diagrams are supplied by the manufacturer of the microprocessor.
- The timing diagrams are essential for a system designer.
- Only from the knowledge of timing diagrams, the matched peripheral devices like memories, ports, etc., can be selected to form a system with a microprocessor as the CPU.

## Memory Read Cycle

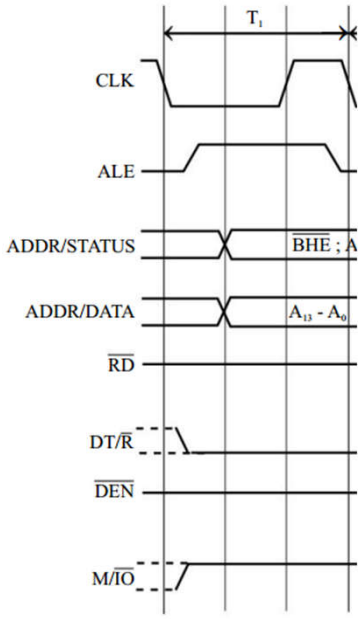
- The memory read cycle is initiated by BIU of 8086 to read a program code or data from memory.
- The normal time taken by memory read cycle is four clock periods.
- The timings of various signals involved in reading a word (16-bit) starting from even address of memory in minimum mode are shown in Fig.






(WR is **high** ; READY is tied **high** permanently or temporarily in the system.)

*Fig : Memory read cycle of 8086.*




**Activities during $T_1$ :**

i) The 8086 outputs a 20-bit memory address on $AD_0$-$AD_{15}$ lines and ADDR/STATUS lines.

ii) The address latch enable signal ALE is asserted **high** in the beginning of $T_1$ and then asserted **low** at the end of $T_1$. This enables the external address latches to latch the address (at the falling edge of ALE) and keep on their output lines.

iii) The direction control signal DT/$\overline{R}$ is asserted **low** to inform the external bidirectional data buffer that the processor has to receive data. (If DT/$\overline{R}$ is already **low** in the previous bus cycle then it remains **low** as such.)

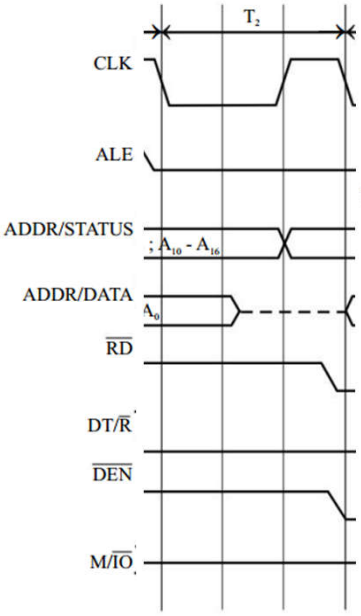
iv) The M/$\overline{IO}$ signal is asserted **high** to indicate memory access. (If M/$\overline{IO}$ is already **high** then it remains **high** as such.)

v) The $\overline{BHE}$ is asserted **low** to enable the odd/upper memory bank.



*Fig : Memory read cycle of 8086.*






**Activities during $T_2$ :**

i) The $AD_0$-$AD_{15}$ lines becomes inactive.

ii) The address is withdrawn from the ADDR/STATUS lines and status signals $S_7$-$S_3$ are issued on these lines. (The $\overline{BHE}$ becomes the status signal $S_7$.)

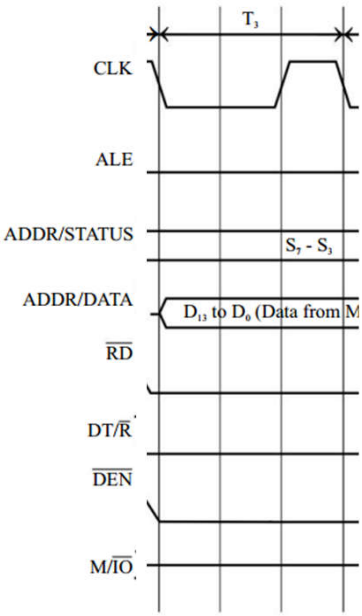
iii) At the end of $T_2$ the read control signal $\overline{RD}$ is asserted **low** to enable the output buffer of memory. The time during which $\overline{RD}$ remains **low** is the time allowed for memory to load data in the data bus.

iv) The $\overline{DEN}$ signal is asserted **low** to enable the external bidirectional data buffers.

v) The 8086 samples READY signal during $T_2$. (If READY is **high** then $T_3$ and $T_4$ are executed otherwise wait states are introduced.)



*Fig : Memory read cycle of 8086.*



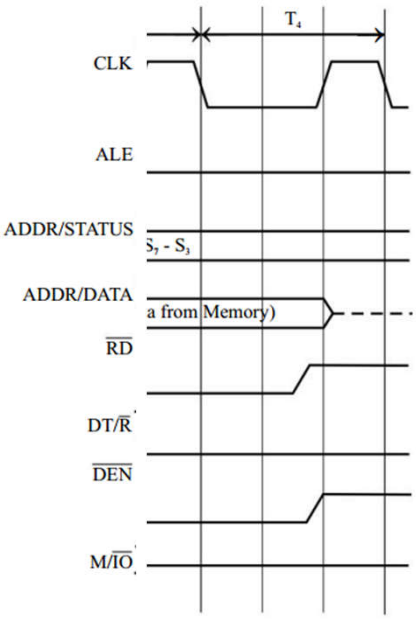

**Activities during $T_3$ :**

No activities are performed during $T_3$. The status of the signals at the end of $T_2$ are maintained throughout $T_3$.



*Fig : Memory read cycle of 8086.*






**Activities during $T_4$ :**

i) The $\overline{RD}$ is asserted **high** and at this time (i.e., at the rising edge of $\overline{RD}$) the data is latched into 8086.

ii) The $\overline{DEN}$ is made **high** to disable the data buffer.

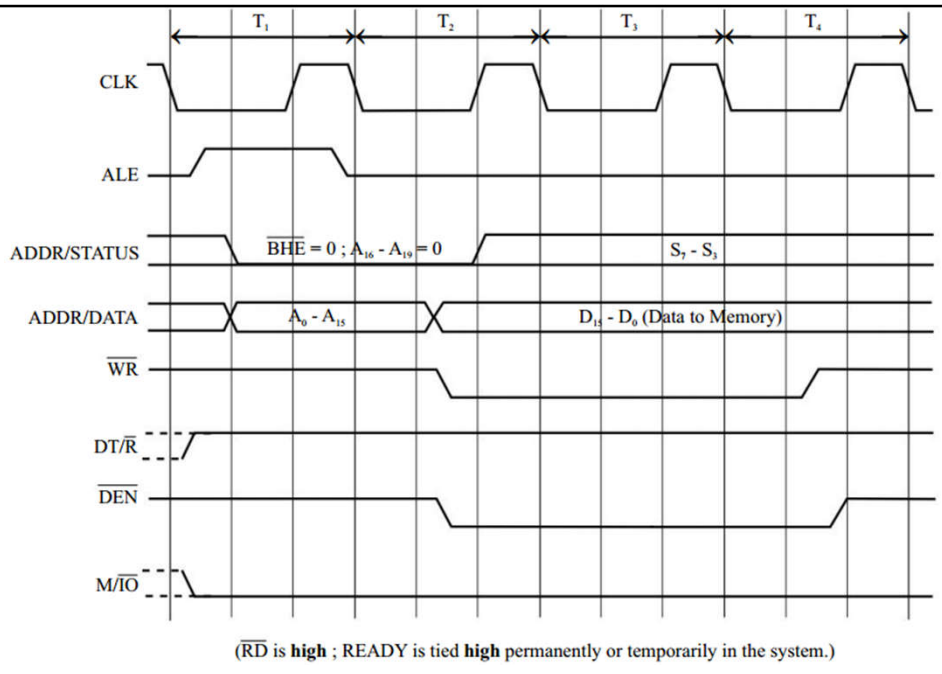


*Fig : Memory read cycle of 8086.*

## Memory Write Cycle

- The memory write cycle is initiated by BIU of 8086 to write a data in memory.
- The normal time taken by memory write cycle is four clock periods.
- The timings of various signals involved in writing a word (16-bit) starting from even address of memory in the minimum mode are shown in Fig.





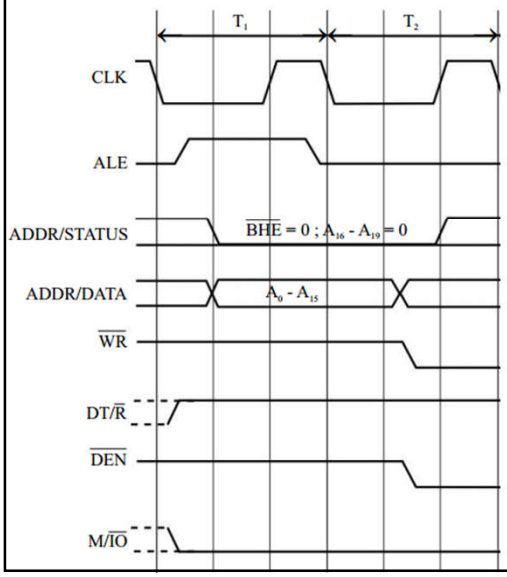

(RD is **high** ; READY is tied **high** permanently or temporarily in the system.)



Fig. : Memory write cycle of 8086.

**Activities during $T_1$ :**

The activities during $T_1$ is same as that of read cycle except DT/$\overline{R}$ signal. In memory write cycle, the DT/$\overline{R}$ signal is asserted **high** to inform the external bidirectional data buffer that the processor is going to transmit data. (If DT/$\overline{R}$ is already **high** in the previous bus cycle then it remains, **high** as such.)

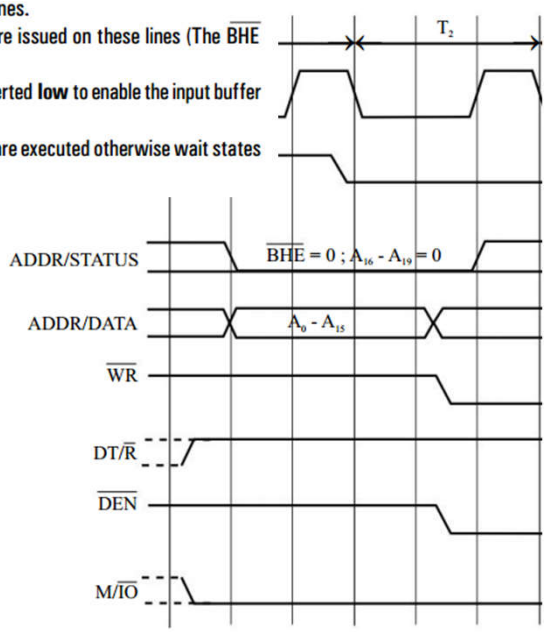



131

Fig. : Memory write cycle of 8086.

**Activities during $T_2$ :**

i) The address is withdrawn from $AD_0$-$AD_{15}$ lines and data is output on these lines.
ii) The address is withdrawn from ADDR/STATUS lines and status signals are issued on these lines (The $\overline{BHE}$ becomes the status signal $S_7$.)
iii) When data is output on data bus, the control signals $\overline{WR}$ and $\overline{DEN}$ are also asserted **low** to enable the input buffer of memory and external data buffer on the data bus respectively.
iv) The 8086 samples READY signal during $T_2$. (If READY is **high** then $T_3$ and $T_4$ are executed otherwise wait states are introduced.)




03-03-2023 16:40:13

Fig. : Memory write cycle of 8086.

**Activities during $T_3$ :**

No activities are performed during $T_3$. The status of the signals at the end of $T_2$ are maintained throughout $T_3$.

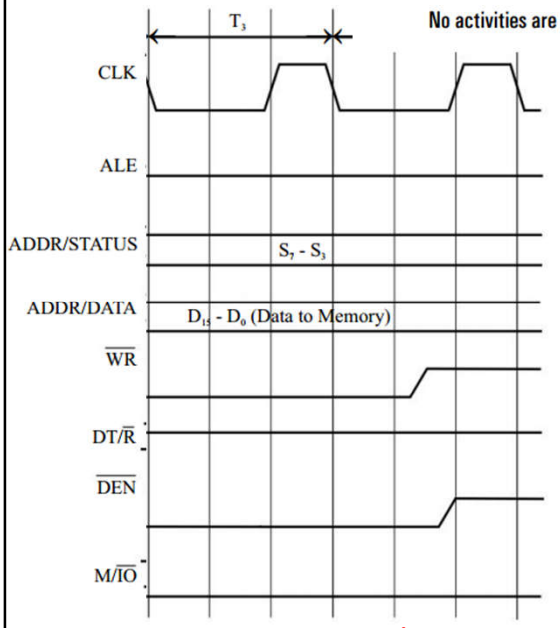



Fig. : Memory write cycle of 8086.

133

**Activities during $T_4$ :**

i) The $\overline{WR}$ is asserted **high** and at this time (i.e., at the rising edge of $\overline{WR}$), the data is latched into memory.
ii) The $\overline{DEN}$ is made **high** to disable the data buffers.

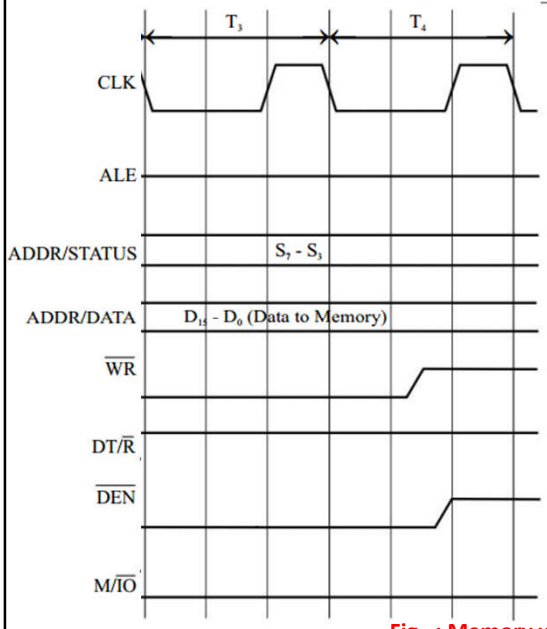


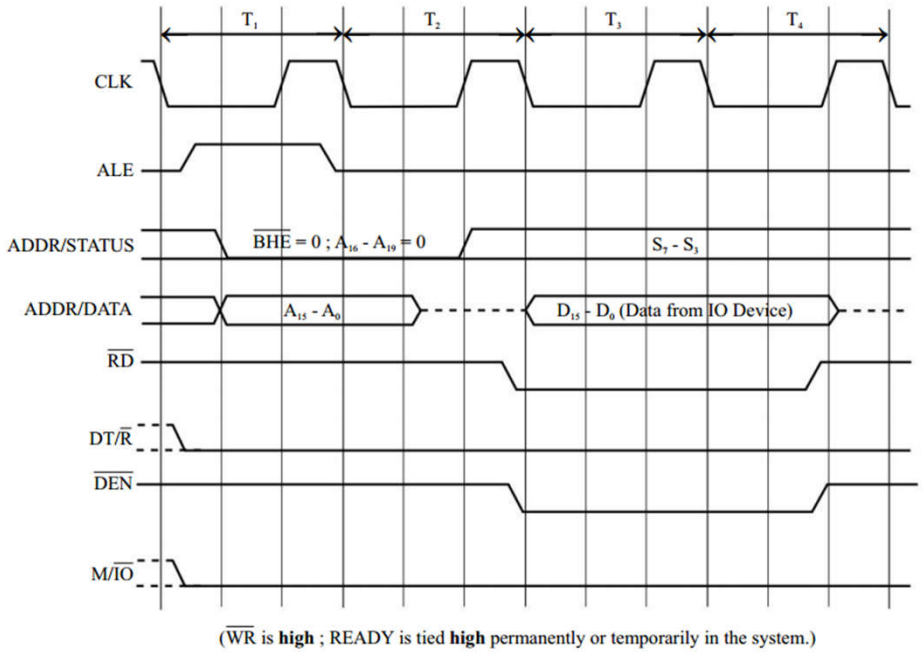

Fig. : Memory write cycle of 8086.

134

## IO Read Cycle

- The IO read cycle is initiated by BIU of 8086 to read a data from IO-mapped device or IO port.
- The normal time taken by IO read cycle is four clock periods.
- The timings of various signals involved in reading an IO port in the minimum mode are shown in Fig.

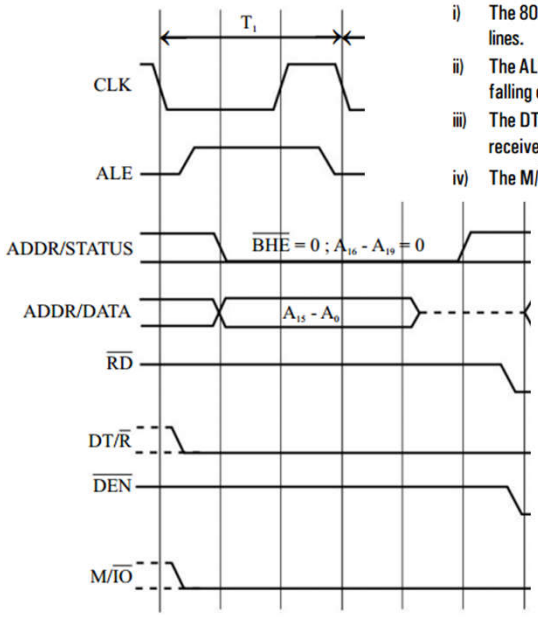





(WR is **high** ; READY is tied **high** permanently or temporarily in the system.)

Fig : IO read cycle of 8086.

**Activities during $T_1$ :**




i) The 8086 outputs a 16-bit IO address on $AD_0$-$AD_{15}$ lines. Logic **low** is output on the $\overline{BHE}$ and ADDR/STATUS lines.

ii) The ALE is asserted **high** and then **low**. This enables the external address latches to latch the address (at the falling edge of ALE) and keep on their output lines.

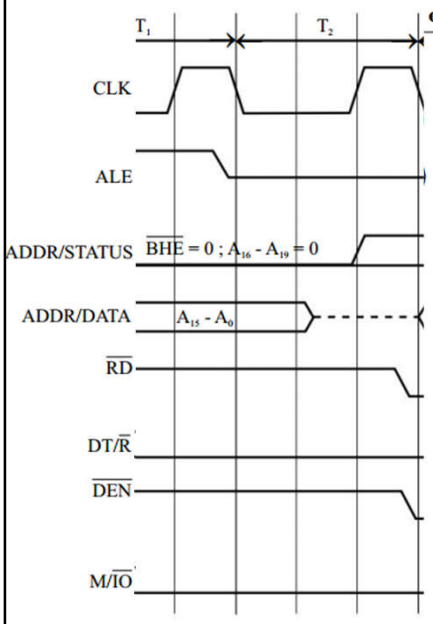
iii) The DT/$\overline{R}$ signal is asserted **low** to inform the external bidirectional data buffer that the processor has to receive data. (If DT/$\overline{R}$ is already **low** in the previous bus cycle then it remains **low** as such.)

iv) The M/$\overline{IO}$ signal is asserted **low** to indicate IO access. (If M/$\overline{IO}$ is already **low** then it remains **low** as such.)



Fig : IO read cycle of 8086.






**during $T_2$ :**

The $AD_0$-$AD_{15}$ lines becomes inactive.

The status signals $S_7$-$S_3$ are issued on ADDR/STATUS lines.

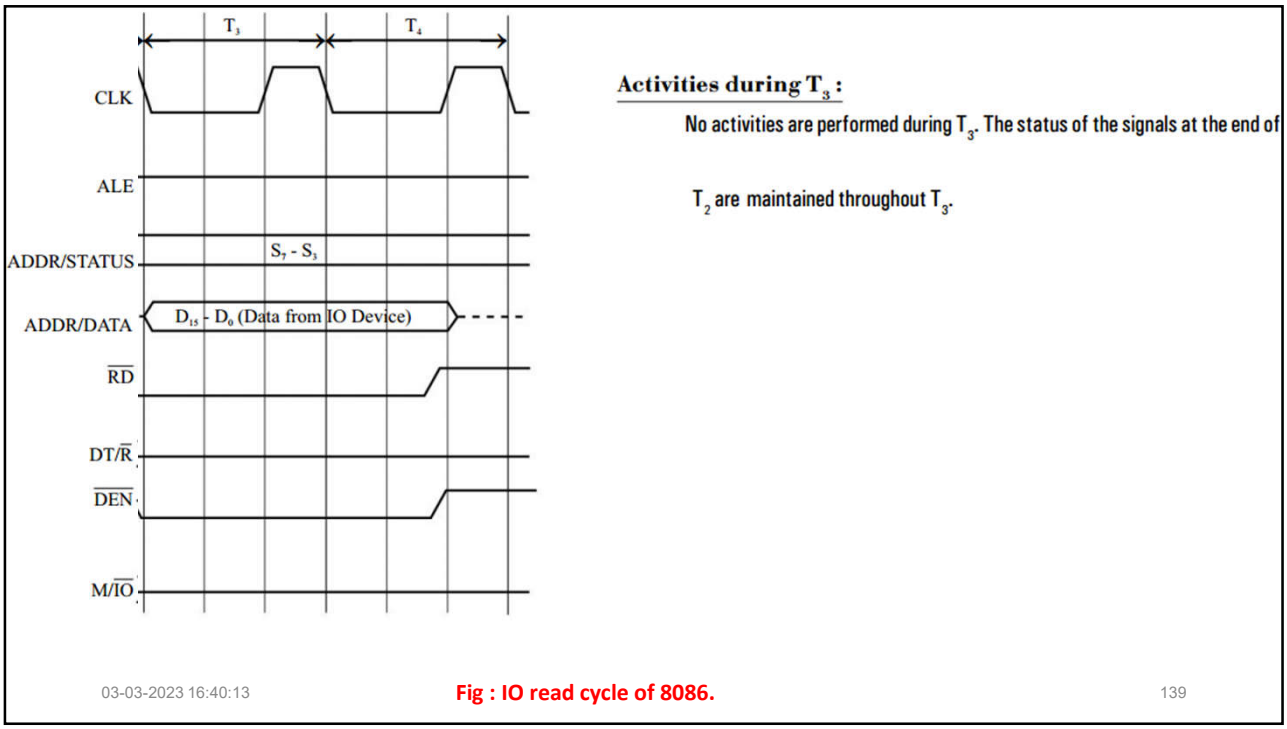
At the end of $T_2$ the read control signal $\overline{RD}$ is asserted **low** to enable the IO device for read operation. The time during which $\overline{RD}$ remains **low** is the time allowed for IO device to load data in the data bus.

The $\overline{DEN}$ signal is asserted **low** to enable the external bidirectional data buffers.

The 8086 samples READY signal during $T_2$. (If READY is **high** then $T_3$ and $T_4$ are executed otherwise wait states are introduced.)



Fig : IO read cycle of 8086.



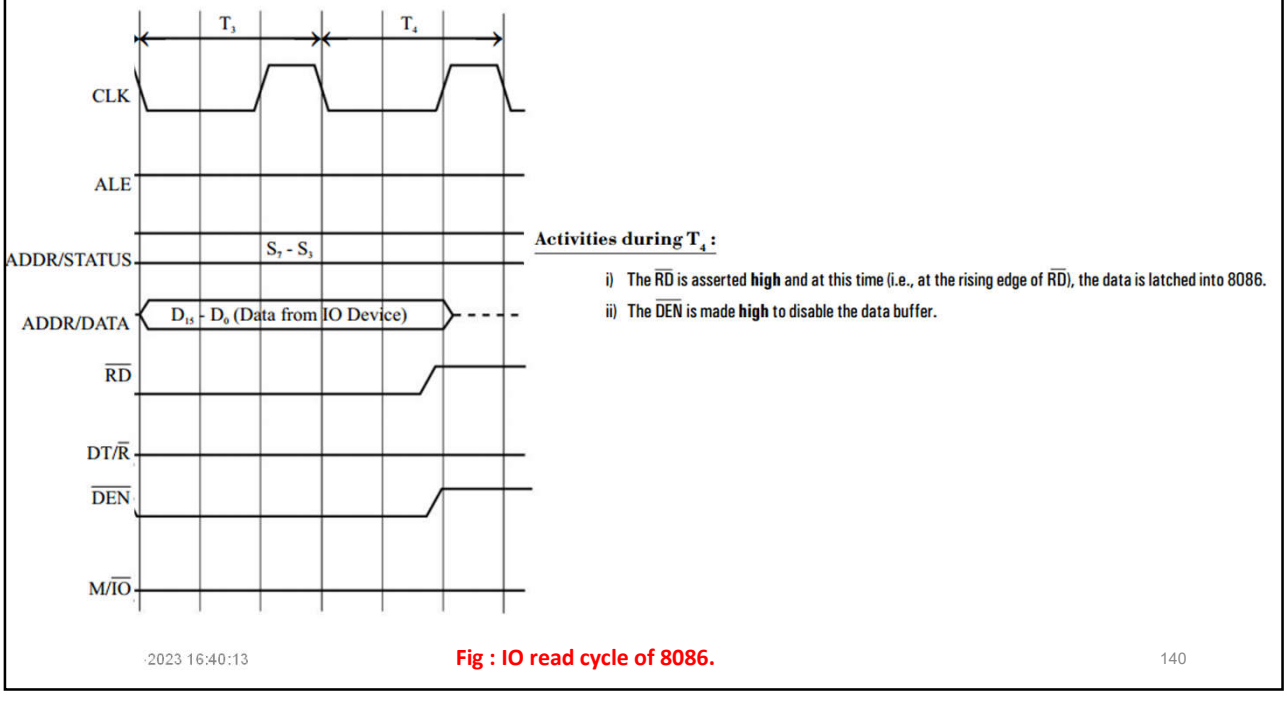

**Activities during $T_3$ :**

No activities are performed during $T_3$. The status of the signals at the end of $T_2$ are maintained throughout $T_3$.



Fig : IO read cycle of 8086.






**Activities during $T_4$ :**

i) The $\overline{RD}$ is asserted **high** and at this time (i.e., at the rising edge of $\overline{RD}$), the data is latched into 8086.

ii) The $\overline{DEN}$ is made **high** to disable the data buffer.

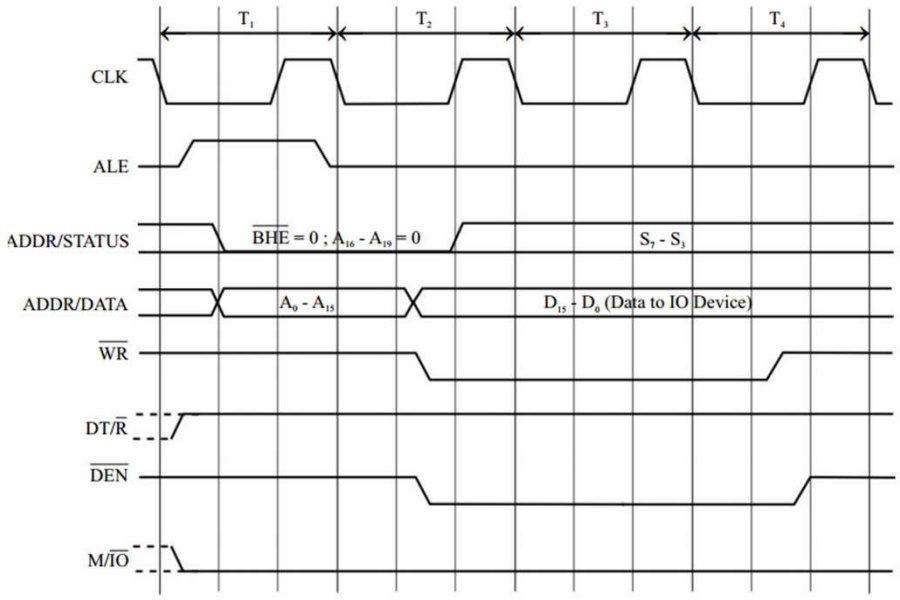


Fig : IO read cycle of 8086.

## IO Write Cycle

- The IO write cycle is initiated by BIU of 8086 to send a data to IO device.
- The normal time taken by IO write cycle is four clock periods.
- The timings of various signals involved in sending a word to IO device in the minimum mode are shown in Fig.





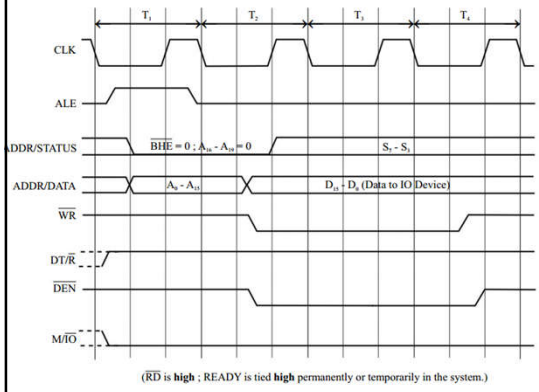

(RD is **high** ; READY is tied **high** permanently or temporarily in the system.)

**Fig. IO write cycle of 8086.**

i) During $T_1$, DT/$\overline{R}$ is asserted **high** to inform the external bidirectional data buffer that the processor is going to transmit data.

ii) During $T_2$, the address is withdrawn from $AD_0$-$AD_{15}$ lines and data is output on these lines. At the same time $\overline{WR}$ is asserted **low** to enable the IO device for write operation and $\overline{DEN}$ is asserted **low** to enable the data buffer on the bus. Here $\overline{RD}$ remains **high**.

iii) During $T_4$, $\overline{WR}$ is asserted **high** and at this time (i.e., at the rising edge of $\overline{WR}$) the data is latched into IO device.




($\overline{RD}$ is **high** ; READY is tied **high** permanently or temporarily in the system.)



Fig. IO write cycle of 8086.



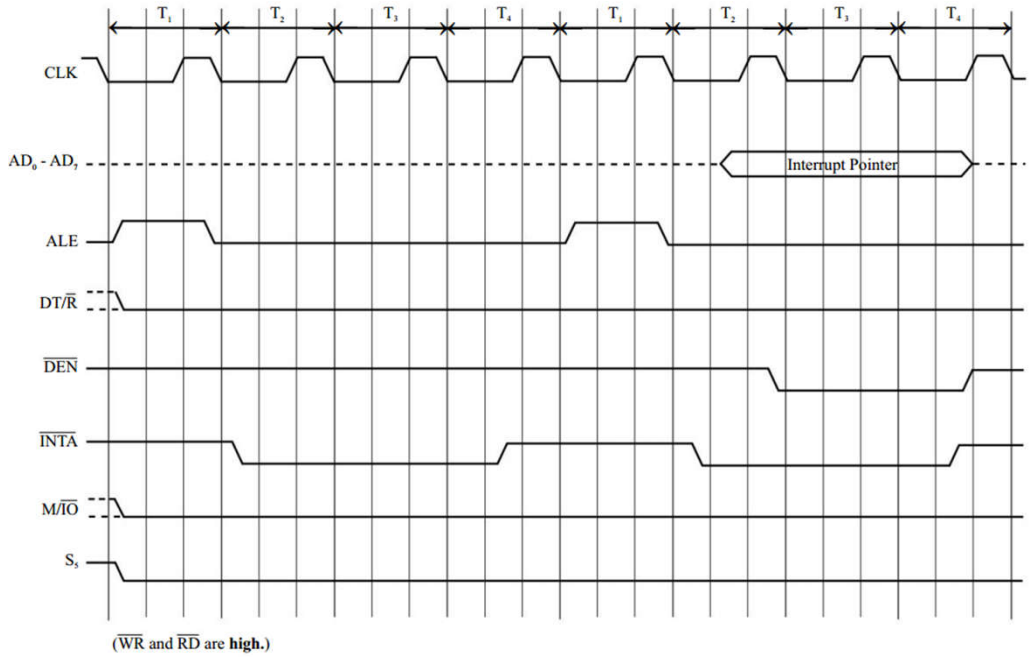
## Interrupt Acknowledge Cycle

- The interrupt acknowledge cycle is executed in response to an interrupt request through the INTR pin of 8086.
- The 8086 samples the status of the INTR pin during the last T-state of instruction (or at the end of instruction execution).
- If INTR is high at the time of sampling and the Interrupt flag is enabled (i.e., IF = 1), then the processor saves (or pushes) the content of the flag register, CS-register, and IP in stack, and clears IF and TF flags, and then executes interrupt acknowledge cycle.

- The time taken by 8086 to execute an interrupt acknowledge cycle is eight T states.
- It is actually two cycles with each cycle extending for 4T states.
- In the first cycle, the processor sends $\overline{\text{INTA}}$ to the interrupting device to inform the acceptance of the interrupt.
- In the second cycle the processor requests the interrupting device to supply an interrupt type number or pointer and read type number from the interrupting device by using $\overline{\text{INTA}}$ signal.



- The timings of various signals during the interrupt acknowledge cycle in minimum mode are shown in Fig.
- During T1 of both the cycles, ALE is made high and low which results in loading a junk value in address latches.




03-03-2023 16:40:13 Fig : Interrupt acknowledge cycle of 8086 147

- During T1 of first cycle DT/$\bar{R}$, M/ $\overline{IO}$ and S5 are asserted low.
- The DT/$\bar{R}$ is asserted low to inform the data buffer that the processor has to receive data. The M/ $\overline{IO}$ is asserted low to indicate IO operation.
- The S5 is asserted low to inform the peripheral devices that the interrupt system is disabled. (Actually S5 is the status of interrupt flag.)

03-03-2023 16:40:13 148

- In both the cycles the $\overline{\text{INTA}}$ is asserted low during T2 and then high during T4.
- In the second cycle, when $\overline{\text{INTA}}$ is low the processor expects an 8-bit interrupt pointer on the lower eight lines (AD0-AD7) of the data bus.
- The time allowed to the interruping device to load the pointer is the time during which $\overline{\text{INTA}}$ remains low.
- The processor samples the interrupt pointer on the rising edge of $\overline{\text{INTA}}$ signal in the second cycle.



Question

1. What are the modes in which 8086 can operate?
2. What is the data and address size in 8086?
3. Explain the function of M/$\overline{\text{IO}}$ in 8086.
4. How is a 20-bit physical address computed in 8086?
5. How is the READY signal used in a microprocessor system?

# INSTRUCTION SET OF 8086



## Instruction Set Of 8086

**The 8086 instructions can be classified into the following six groups.**

1. Data transfer instructions
2. Arithmetic instructions
3. Logical instructions
4. String manipulating instructions
5. Control transfer instructions
6. Processor control instructions

- The **data transfer group** includes instructions for **moving data** between registers, register and memory, register and stack memory, and accumulator & IO device.
- The arithmetic group includes instructions for the addition and subtraction of binary, BCD, and ASCII data, and instructions for multiplication and division of signed and unsigned binary data.



- The logical group includes instructions for performing **logical operations like AND, OR, Exclusive-OR, Complement, Shift, Rotate,** etc.
- The string manipulation group includes instructions for **moving string data between two memory locations and comparing string data** word by word or byte by byte.

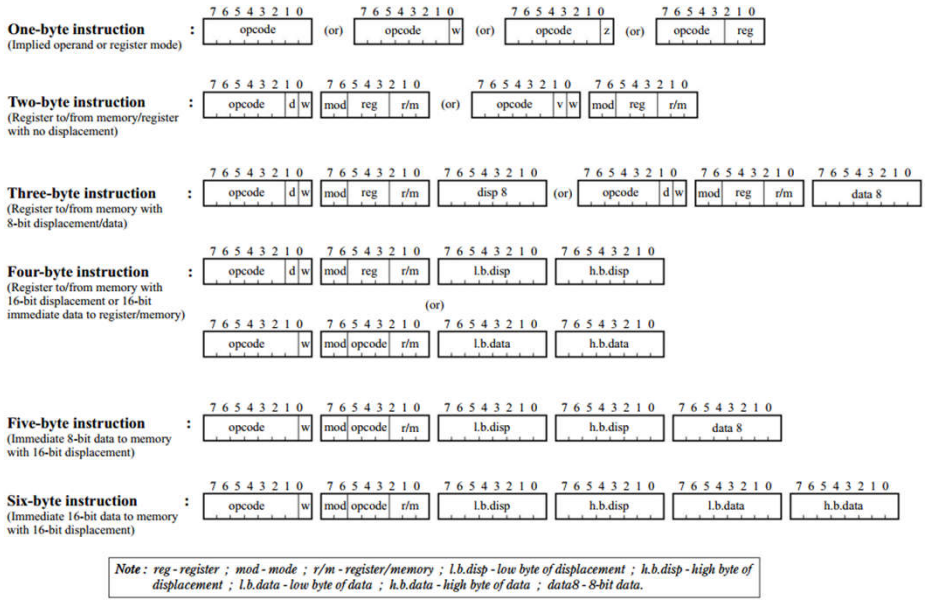
- The control transfer group includes instructions to call a procedure/subroutine in the main program.
- It also includes instructions to jump from one part of a program to another part either **conditionally (after checking flags) or unconditionally (without checking flags).**
- The processor control group includes instructions **to set/clear the flags, to delay and halt the processor execution.**



## INSTRUCTIONS FORMAT

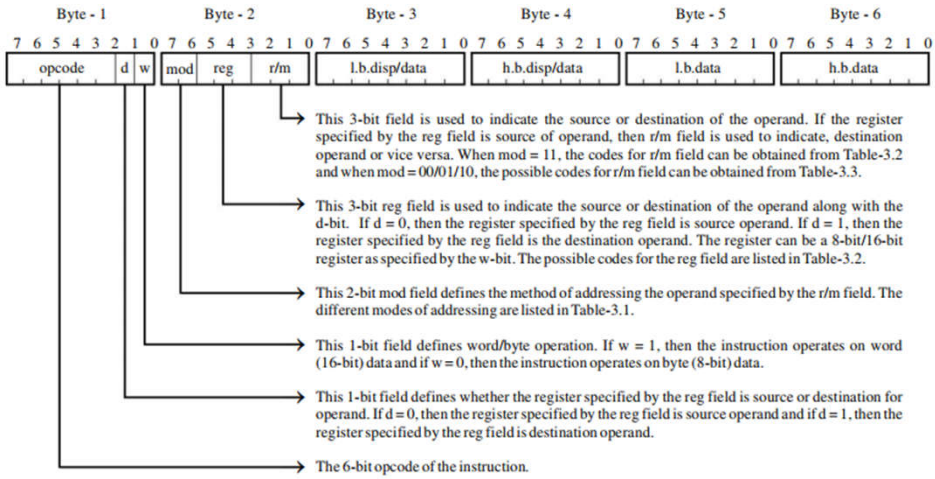
## INSTRUCTIONS FORMAT

The size of 8086 instruction is one to six bytes. Some examples of 8086 instruction formats are shown in Fig.




*Note : reg - register ; mod - mode ; r/m - register/memory ; l.b.disp - low byte of displacement ; h.b.disp - high byte of displacement ; l.b.data - low byte of data ; h.b.data - high byte of data ; data8 - 8-bit data.*

157




*Note : l.b.disp/data - low byte displacement or data ; h.b. disp/data - high byte displacement or data ; l.b. data - low byte data ; h.b. data - high byte data.*

**The General format of 8086 instruction**

03-03-2023 16:40:13

158

TABLE - : CODES FOR mod FIELD

| Code for mod field | Name of the mode |
|---|---|
| 00 | Memory mode with no displacement |
| 01 | Memory mode with 8-bit signed displacement |
| 10 | Memory mode with 16-bit unsigned displacement |
| 11 | Register mode |

TABLE - : CODES FOR reg FIELD

| Code for reg field | Name of the register represented by the code when w = 0 or 1 | |
|---|---|---|
| | When w = 0 | When w = 1 |
| 000 | AL | AX |
| 001 | CL | CX |
| 010 | DL | DX |
| 011 | BL | BX |
| 100 | AH | SP |
| 101 | CH | BP |
| 110 | DH | SI |
| 111 | BH | DI |



- In general, the first byte of the instruction will have a 6-bit opcode and two special bit indicators d-bit and w-bit or (s-bit and w-bit) or (v-bit and w-bit).
- Some instructions will have an 8-bit opcode and some instructions will have a 7-bit opcode followed by special bit indicator w-bit or z-bit.
- Some of the instructions will have a 2-bit or 3-bit register field in the first byte of the instruction.

The usage of special one-bit indicators are given below:

w-bit : This bit appears in the format of instructions which can operate on both byte and word data.

If w = 0, then the data operated by the instruction is 8-bit/byte.

If w = 1, then the data operated by the instruction is 16-bit/word.



d-bit : This bit appears in the format of instructions which has a double operand.

- In double operand instructions, one of the operand should be a register specified by reg field.
- The d-bit is used to specify whether the register specified by reg field is source operand or destination operand.

  If d = 0, then the register specified by reg field is source operand.

  If d = 1, then the register specified by reg field is destination operand.

s-bit : This bit appears in the format of arithmetic instructions which operate on immediate data. If s = 1, w = 1 and immediate data is 8-bit then the immediate data is sign extended to 16-bit and used for arithmetic operation.

sw = 00 → 8-bit operation with an 8-bit immediate data.

sw = 01 → 16-bit operation with a 16-bit immediate data.

sw = 11 → 16-bit operation with a sign extended 8-bit immediate operand.



v-bit : This bit appears in the format of shift and rotate instructions.

If v = 0, then the shift/rotate operation is performed one time.

If v = 1, then the content of CL is count value for number of shift/rotate operations.

z-bit : This bit appears in the format of REP prefix for string instructions and is used for comparing with zero flag.

If z = 0, then repeat execution of string instruction until zero flag is zero.

If z = 1, then repeat execution of string instruction until zero flag is one.

- In multi-byte instructions, the second byte will specify the addressing mode of the operands.
- The second byte usually has three fields : mod, reg and r/m.
- The mod field is 2-bit wide and it defines the method of addressing the operand specified by r/m field.
- The r/m field is 3-bit wide and it is used to indicate the source or destination operand in memory/register.



- The reg field is 3-bit wide and it is used to indicate the source or destination operand in register.
- If register specified by reg field is source operand then r/m field is used to indicate destination operand or vice versa.
- The mod and r/m field are used to calculate the effective address of memory operand as shown in Table

## TABLE - : CODES FOR r/m FIELD

| Code for r/m field | Effective address calculation when mod = 00/01/10 | | |
|---|---|---|---|
| | mod = 00 | mod = 01 | mod = 10 |
| 000 | [BX + SI] | [BX + SI + disp8] | [BX + SI + disp16] |
| 001 | [BX + DI] | [BX + DI + disp8] | [BX + DI + disp16] |
| 010 | [BP + SI] | [BP + SI + disp8] | [BP + SI + disp16] |
| 011 | [BP + DI] | [BP + DI + disp8] | [BP + DI + disp16] |
| 100 | [SI] | [SI + disp8] | [SI + disp16] |
| 101 | [DI] | [DI + disp8] | [DI + disp16] |
| 110 | [disp16] | [BP + disp8] | [BP + disp16] |
| 111 | [BX] | [BX + disp8] | [BX + disp16] |

*Note : disp 8 → 8-bit signed displacement.*
*disp16 → 16-bit unsigned displacement.*



**TABLE - : MEMORY ADDRESS CALCULATION IN 8086 USING DEFAULT SEGMENT REGISTER**

| S.No. | Addressing mode | Effective address EA | Physical address $MA/MA_S$ |
|---|---|---|---|
| 1. | [BX + SI] | EA = (BX) + (SI) | MA = (DS) × $16_{10}$ + EA |
| 2. | [BX + SI + disp8] | disp8 $\xrightarrow{\text{sign extend}}$ disp16<br>EA = (BX) + (SI) + disp16 | MA = (DS) × $16_{10}$ + EA |
| 3. | [BX + SI + disp16] | EA = (BX) + (SI) + disp16 | MA = (DS) × $16_{10}$ + EA |
| 4. | [BX + DI] | EA = (BX) + (DI) | MA = (DS) × $16_{10}$ + EA |
| 5. | [BX + DI + disp8] | disp8 $\xrightarrow{\text{sign extend}}$ disp16<br>EA = (BX) + (DI) + disp16 | MA = (DS) × $16_{10}$ + EA |

- In multi-byte instructions the bytes following the opcode and address mode bytes (1st and 2nd bytes) may be any one of the following:
- i) no additional bytes.
- ii) two-byte effective address.
- iii) one-byte (8-bit) signed displacement or two-byte (16-bit) unsigned displacement.



- iv) one-byte (8-bit) immediate data or two-byte (16-bit) immediate data (operand).
- v) one/two byte displacement followed by one/two-byte immediate data (operand).

vi) two-byte effective address followed by two-byte segment address.

*Note : If a displacement or immediate data is two bytes long, then the low order byte always appears first.*

## ADDRESSING MODES OF 8086



## ADDRESSING MODES OF 8086

- Every instruction of a program has to operate on data.
- The method of specifying the data to be operated by the instruction is called addressing.
- The 8086 has 12 addressing modes, and they can be classified into the following five groups.

| | | |
|---|---|---|
| 1. Register addressing<br>2. Immediate addressing | Group I | : Addressing modes for register and immediate data |
| 3. Direct addressing<br>3. Register indirect addressing<br>5. Based addressing<br>6. Indexed addressing<br>7. Based index addressing<br>8. String addressing | Group II | : Addressing modes for memory data |
| 9. Direct IO port addressing<br>10. Indirect IO port addressing | Group III | : Addressing modes for IO ports |
| 11. Relative addressing | Group IV | : Relative addressing mode |
| 12. Implied addressing | Group V | : Implied addressing mode |



Note :

1. The “register” or “register + constant” enclosed by square brackets in the operand field of instructions refer to **the method of effective address calculation of memory.**

- The 16 - bit constant enclosed by square brackets in the operand field of instructions refer **to the effective address of memory data.**
- The 8-bit/16-bit constants which are not enclosed by square brackets in the operand field refer to immediate data.

2. The term MA used in the symbolic description of instructions refer to physical memory address of data segment memory and MASrefer to physical memory address of stack segment memory and MAE refer to physical memory address of extra segment memory.

3. The register/memory enclosed by brackets in symbolic description refer to the content of register/memory.

4. For hexa-decimal constant (data/address) the letter H is included at the end of 8-bit/16-bit constants(data/address), and the numeral 0 is included in the front of hexadecimal constant starting with A through F.



## Register Addressing

In **register addressing,** the instruction will specify the name of the register which holds the data to be operated by the instruction.

***Examples :***

**a) MOV CL,DH** **(CL) ← (DH)**

*The content of 8-bit register DH is moved to another 8-bit register CL.*

**b) MOV BX,DX** **(BX) ← (DX)**

*The content of 16-bit register DX is moved to another 16-bit register BX.*

**Immediate Addressing:** In immediate addressing mode, an 8-bit or 16-bit data is specified as part of the instruction.

***Examples :***

**a) MOV DL,08H** **(DL) ← $08_H$**

*The 8-bit data ($08_H$) given in the instruction is moved to DL-register.*

**b) MOV AX,0A9FH** **(AX) ← $0A9F_H$**

*The 16-bit data ($0A9F_H$) given in the instruction is moved to AX-register.*



## Direct Addressing

- In direct addressing, an unsigned 16-bit displacement or signed 8-bit displacement will be specified in the instruction.
- The displacement is the Effective Address(EA) or offset.
- In case of 8-bit displacement, the effective address is obtained by sign extending the 8-bit displacement to 16-bit.

- The 20-bit physical address of memory is calculated by multiplying the content of DS-register by $16_{10}$ (or 10H ) and adding to effective address.
- When segment override prefix is employed, the content of segment register specified in the override prefix will be used for segment base address calculation instead of DS-register.



***Examples :***

**a) MOV DX,[08H]**

**EA = $0008_H$ (sign extended 8-bit displacement)**

**BA = (DS) $\times 16_{10}$ ; MA = BA + EA**

**(DX) ← (MA) or DL ← (MA)**

**DH ← (MA+1)**

- *The Effective Address(EA) is obtained by sign extending the 8-bit displacement given in the instruction to 16-bit.*
- *The segment Base Address(BA) is computed by multiplying the content of DS by $16_{10}$.*
- *The Memory Address(MA) is computed by adding the Effective Address (EA) to segment Base Address(BA).*
- *The content of memory whose address is calculated as explained above is moved to DL-register and the content of next memory location is moved to DH-register.*

b) MOV AX, [089DH]

EA = $089D_H$ ; BA = (DS) × $16_{10}$ ; MA = BA + EA

(AX) ← (MA) or (AL) ← (MA)

(AH) ← (MA+1)

- Here the 16-bit displacement given in the instruction is the effective address.
- The segment **Base Address(BA) is computed by multiplying the content of DS by $16_{10}$.**
- The Memory Address(MA) is computed by adding the Effective Address(EA) to segment Base Address(BA).
- The content of memory whose address is calculated as explained above is moved to AL-register and the content of next memory location is moved to AH-register.



**Register Indirect Addressing**

- In register indirect addressing, the name of the register which holds the Effective Address(EA) will be specified in the instruction.
- The register used to hold the effective address are BX, SI and DI.
- The content of DS is used for segment base address calculation.
- When segment override prefix is employed, the content of segment register specified in the override prefix will be used for base address calculation instead of DS-register.
- The base address is obtained by multiplying the content of segment register by 16 10.
- The 20-bit physical address of memory is computed by adding the effective address to base address.

***Examples :***

**a) MOV CX, [BX]**

**EA = (BX) ; BA = (DS) × $16_{10}$ ; MA = BA + EA**

**(CX) ← (MA) or (CL) ← (MA)**

**(CH) ← (MA+1)**

*The content of BX is the **E**ffective **A**ddress(EA). The segment **B**ase **A**ddress(BA) is computed by multiplying the content of DS by $16_{10}$. The **M**emory **A**ddress(MA) is obtained by adding BA and EA.*

*The content of memory whose address is calculated as explained above is moved to CL-register and the content of next memory location is moved to CH-register.*



**b) MOV AX,[SI]**

**EA = (SI) ; BA = (DS) × $16_{10}$ ; MA = BA + EA**

**(AX) ← (MA) or (AL) ← (MA)**

**(AH) ← (MA + 1)**

*The content of SI is the **E**ffective **A**ddress (EA). The segment **B**ase **A**ddress(BA) is computed by multiplying the content of DS by $16_{10}$. The memory address is obtained by adding BA and EA.*

*The content of memory whose address is calculated as explained above is moved to AL-register and the content of next memory location is moved to AH-register.*

**Based Addressing**

- In this addressing mode, the BX or BP-register is used to hold a base value for effective address and a signed 8-bit or unsigned 16-bit displacement will be specified in the instruction.
- The displacement is added to base value in BX or BP to obtain the Effective Address(EA).
- In case of 8-bit displacement, it is sign extended to 16-bit before adding to base value.
- When BX is used to hold base value for EA, the 20-bit physical address of memory is calculated by multiplying the content of DS by $16_{10}$ adding to EA.
- When BP is used to hold base value for EA, the 20-bit physical address of memory is calculated by multiplying the content of SS by $16_{10}$ and adding to EA.



***Example :***

**MOV AX, [BX+08H]**

**$0008_H$ $08_H$ ; EA = (BX) + $0008_H$**

**BA = (DS) × $16_{10}$ ; MA = BA + EA**

**(AX) ← (MA) or (AL) ← (MA)**

**(AH) ← (MA+1)**

*The effective address is calculated by sign extending the 8-bit displacement given in the instruction to 16-bit and adding to the content of BX-register. The **B**ase **A**ddress(BA) is obtained by multiplying the content of DS by $16_{10}$. The **M**emory **A**ddress(MA) is obtained by adding BA and EA.*

*The content of memory whose address is calculated as explained above is moved to AL-register and the content of next memory is moved to AH-register.*

**Indexed Addressing**

- In this addressing mode, the SI or DI-register is used to hold an index value for memory data and a signed 8-bit displacement or unsigned 16-bit displacement will be specified in the instruction.
- The displacement is added to index value in SI or DI-register to obtain the Effective Address (EA).
- In case of 8-bit displacement it is sign extended to 16-bit before adding to index value.
- The 20-bit memory address is calculated by multiplying the content of Data Segment (DS) by $16_{10}$ and adding to EA.

*Note : In general the effective address = Reference + modifier.*
*In this context, the based and indexed addressing looks similar, but in based addressing, base value is the reference and displacement is the modifier, whereas in indexed addressing, displacement is the reference and index value is the modifier.*



***Example :***

**MOV CX, [SI+0A2H]**

$FFA2_H \xleftarrow{\text{sign extend}} A2_H$ ; $EA = (SI) + FFA2_H$
$BA = (DS) \times 16_{10}$ ; $MA = BA + EA$
$(CX) \leftarrow (MA)$ or $(CL) \leftarrow (MA)$
$(CH) \leftarrow (MA + 1)$

*The effective address is calculated by sign extending the 8-bit displacement given in the instruction to 16-bit and adding to the content of SI-register. The **B**ase **A**ddress (BA) is obtained by multiplying the content of DS by $16_{10}$. The **M**emory **A**ddress(MA) is obtained by adding BA and EA.*

*The content of memory whose address is calculated as explained above is moved to CL-register and the content of next memory is moved to CH-register.*

**Based Indexed Addressing**

- In this addressing mode, the effective address is given by sum of base value, index value and an 8-bit or 16-bit displacement specified in the instruction.
- The base value is stored in BX or BP-register. The index value is stored in SI or DI-register.
- In case of 8-bit displacement, it is sign extended to 16-bit before adding to base value.
- This type of addressing will be useful in addressing two dimensional arrays where we require two modifiers.



- *When BX is used to hold base value for EA, the 20-bit physical address of memory is calculated by multiplying the content of DS by $16_{10}$ and adding to EA.*
- *When BP is used to hold the base value for EA, the 20-bit physical address of memory is obtained by multiplying the content of SS-register by $16_{10}$ and adding it to EA.*

***Example :***

**MOV DX, [BX+SI+0AH]**

$000A_H \xleftarrow{\text{sign extend}} 0A_H$ ; EA = (BX) + (SI) + $000A_H$

BA = (DS) × $16_{10}$ ; MA = BA + EA

(DX) ← (MA) or (DL) ← (MA)

(DH) ← (MA + 1)

*The **E**ffective **A**ddress(EA) is calculated by sign extending the 8-bit displacement given in the instruction to 16-bit and adding it to the content of BX and SI-register. The **B**ase **A**ddress(BA) is obtained by multiplying the content of DS by $16_{10}$. The 20-bit **M**emory **A**ddress(MA) is obtained by adding BA and EA.*

*The content of memory whose address is calculated as explained above is moved to DL-register and the content of the next memory location is moved to DH-register.*



**STRING ADDRESSING**

- This addressing mode is employed in string instructions to operate on string data.
- In string addressing mode, the Effective Address(EA) of source data is stored in SI-register and the EA of destination data is stored in DI-register.
- The segment register used for calculating base address for source data is DS and can be overridden.
- The segment register used for calculating base address for destination is ES and cannot be overridden.

- This addressing mode also supports auto increment/decrement of index registers SI and DI depending on Direction Flag(DF).

- If DF=1, then the content of index registers are decremented to point to next byte/word of the string after execution of a string instruction.
- If DF=0, then the content of index registers are incremented to point to previous byte/word of the string after execution of a string instruction.

(For word operand, the content of index registers are incremented/decremented by two and for byte operand, the content of index registers are incremented/decremented by one .)



***Example :***

**MOVS BYTE**

**EA = (SI) ; BA = (DS) × $16_{10}$ ; MA = BA + EA**

**$EA_E$ = (DI) ; $BA_E$ = (ES) × $16_{10}$ ; $MA_E$ = $BA_E$ + $EA_E$**

**($MA_E$) ← (MA)**

**If DF = 1, then (SI) ← (SI) – 1 and (DI) ← (DI) – 1**

**If DF = 0, then (SI) ← (SI) + 1 and (DI) ← (DI) + 1**

*This instruction move a byte of string data from one memory location to another memory location. The address of source memory location is calculated by multiplying the content of DS by $16_{10}$ and adding to SI. The address of destination memory location is calculated by multiplying the content of ES by $16_{10}$ and adding to DI.*

*After the move operation if DF = 1, then the content of index registers DI and SI are decremented by one. If DF = 0, then the content of index registers DI and SI are incremented by one.*

## Direct IO Port Addressing

- This addressing mode is used to access data from standard IO-mapped devices or ports.
- In the direct port addressing mode, an 8-bit port address is directly specified in the instruction.

***Example :***

**IN AL, [09H]**

$PORT_{addr} = 09_H$

**(AL) ← (PORT)**

*The content of the port with address $09_H$ is moved to AL-register.*



## Indirect IO Port Addressing

- This addressing mode is used to access data from standard IO-mapped devices or ports.
- In the indirect port addressing mode, the instruction will specify the name of the register which holds the port address.
- In 8086, the 16-bit port address is stored in DX-register

***Example :***

**OUT [DX], AX**

$PORT_{addr}$ **= (DX) ; (PORT) ← (AX)**

*The content of AX is moved to the port whose address is specified by DX-register.*

**Relative Addressing**

- In this addressing mode the effective address of a program instruction is specified relative to the Instruction Pointer (IP) by an 8-bit signed displacement.

***Example :***

**JZ 0AH**

**$000A_H$ ← sign extend $0A_H$**

**If ZF = 1, then (IP) ← (IP) + $000A_H$ ; $EA_C$ = (IP) + $000A_H$**

**$BA_C = (CS) \times 16_{10}$ ; $MA_C = BA_C + EA_C$**

***Note : Suffix C refers to code memory***

*If ZF = 1, then the program control jumps to a new code address as calculated above. If ZF = 0, then the next instruction of the program is executed.*



**Implied Addressing**

In implied addressing mode, the instruction itself will specify the data to be operated by the instruction.

***Example :***

**CLC - Clear carry ; CF ← 0**

Execution of this instruction will clear the **C**arry **F**lag(CF).

INSTRUCTION SET OF 8086

The 8086/8088 instructions are categorised into the following main types.

(i)

Data Copy/Transfer Instructions

These types of instructions are used to transfer data from

source operand to destination operand. All the store, move, load, exchange, input and output instructions belong to this category.

(ii)

Arithmetic and Logical Instructions

All the instructions performing arithmetic, logical, increment, decrement, compare and scan instructions belong to this category.



(iii)

Branch Instructions

These instructions transfer control of execution to the specified address.

All the call, jump, interrupt and return instructions belong to this class

(iv)

Loop Instructions

If these instructions have REP prefix with CX used as count register, they can be used to implement unconditional and conditional loops. The LOOP, LOOPNZ and LOOPZ instructions belong to this category.

These are useful to implement different loop structures.

(v)

Machine Control Instructions

These instructions control the machine status. NOP, HLT, WAIT and LOCK instructions belong to this class.

(vi)

Flag Manipulation Instructions

All the instructions which directly affect the flag register, come under this group of instructions. Instructions like CLD,STD,CLI,STI, etc. belong to this category of instructions.



(vii)

Shift and Rotate Instructions

These instructions involve the bitwise shifting or rotation in either direction with or without a count in CX.

(viii)

String Instructions

These instructions involve various string manipulation operations like load, move, scan, compare, store, etc. These instructions are only to be operated upon the strings.

## Data Copy/Transfer Instructions



## Data Copy/Transfer Instructions

MOV: Move

- This data transfer instruction transfers data from one register/memory location to another register/memory location.
- The source may be any one of the segment registers or other general or special purpose registers or a memory location and, another register or memory location may act as destination.

- In case of immediate addressing mode, a segment register cannot be a destination register.
- In other words, direct loading of the segment registers with immediate data is not permitted.
- To load the segment registers with immediate data, one will have to load any general purpose register with the data and then it will have to be moved to that particular segment register.



Load DS with 5000H.

1. MOV DS, 5000H; Not permitted (invalid)
   Thus to transfer an immediate data into the segment register, the correct procedure is given below.
2. MOV AX, 5000H
   MOV DS, AX
   It may be noted here that both the source and destination operands cannot be memory locations (except for string instructions). Other MOV instruction examples are given below with the corresponding addressing modes.
3. MOV AX, 5000H; Immediate
4. MOV AX, BX; Register
5. MOV AX, [SI]; Indirect
6. MOV AX, [2000H]; Direct
7. MOV AX, 50H[BX]; Based relative, 50H Displacement

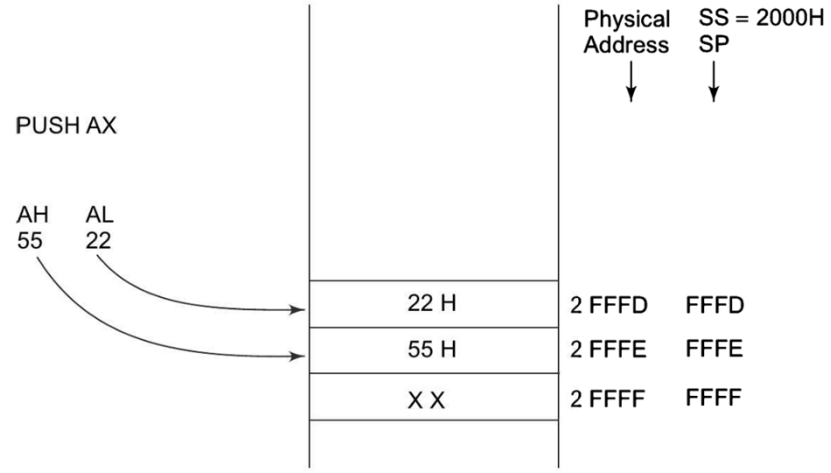
**PUSH: Push to Stack**

- This instruction pushes the contents of the specified register/memory location on to the stack.
- The stack pointer is decremented by 2, after each execution of the instruction.
- The actual current stack-top is always occupied by the previously pushed data.



- Hence, the push operation decrements SP by two and then stores the two byte contents of the operand onto the stack.
- **The higher byte is pushed first and then the lower byte.**
- Thus out of the two decremented stack addresses **the higher byte occupies the higher address and the lower byte occupies the lower address.**

AH, AL contains data to be pushed.




Pushing Data to Stack Memory



The sequence of operation as below:

1. Current stack top is already occupied so decrement SP by one then store AH into the address pointed to by SP.
2. Further decrement SP by one and store AL into the location pointed to by SP.

Thus SP is decremented by 2 and AH-AL contents are stored in stack memory as shown in Fig.

Contents of SP points to a new stack top.

Example

1. PUSH AX
2. PUSH DS
3. PUSH [5000H]; Content of location 5000H and 5001H in DS are pushed onto the stack



## POP: Pop from Stack

- This instruction when executed, loads the specified register/memory location with the contents of the memory location of which the address is formed using the current stack segment and stack pointer as usual.
- The stack pointer is incremented by 2. The POP instruction serves exactly opposite to the PUSH instruction.

- 16-bit contents of current stack top are poped into the specified operand as follows.

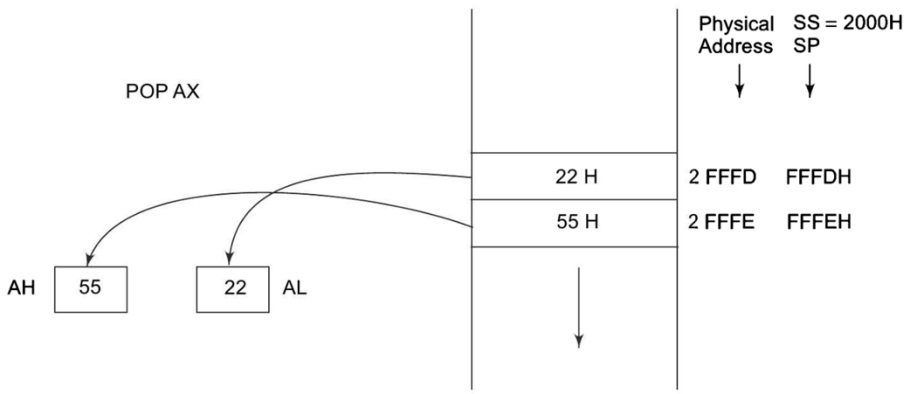
The sequence of operation is as below.

1. Contents of stack top memory location is stored in AL and SP is incremented by one

2. Further contents of memory location pointed to by SP are copied to AH and SP is again incremented by 1

**Effectively SP is incremented by 2 and points to next stack top.**






***Popping Register Contents from Stack Memory***

Example
1 . POP AX
2 . POP DS
3 . POP [ 5000H]



XCHG: Exchange

- This instruction exchanges the contents of the specified source and destination operands, which may be registers or one of them may be a memory location.
- However, exchange of contents of two memory locations is not permitted. Immediate data is also not allowed in these instructions.

Example

**1 . XCHG [5000 H ] ,AX**

;This instruction exchanges data between AX and a memory location [5000H] in the data segment.

**2. XCHG BX , AX**

;This instruction exchanges data between AX and BX.



IN: Input the Port

- This instruction is used for reading an input port. The address of the input port may be specified in the instruction directly or indirectly.
- AL and AX are the allowed destinations for 8 and 16-bit input operations.
- DX is the only register (implicit) which is allowed to carry the port address.
- If the port address is of 16 bits it must be in DX. The examples are given as shown:

```
1. IN AL,03H        ; This instruction reads data from an 8-bit port whose address
                    ; is 03H and stores it in AL.
2. IN AX, DX        ; This instruction reads data from a 16-bit port whose
                    ; address is in DX (implicit) and stores it in AX.
3. MOV DX,0800H     ; The 16-bit address is taken in DX.
   IN AX, DX        ; Read the content of the port in AX.
```



**OUT: Output to the Port**

- This instruction is used for writing to an output port.
- The address of the output port may be specified in the instruction directly or implicitly in DX.
- Contents of AX or AL are transferred to a directly or indirectly addressed port after execution of this instruction.

- The data to an odd addressed port is transferred on D8-D15 while that to an even addressed port is transferred on D0-D7.
- The registers AL and AX are the allowed source operands for 8-bit and 16-bit operations respectively.
- If the port address is of 16 bits it must be in DX.



Example

```
1. OUT    03H,AL   ; This sends data available in AL to a port whose
                   ; address is 03H.
2. OUT    DX, AX   ; This sends data available in AX to a port whose
                   ; address is specified implicitly in DX.
3. MOV DX, 0300H ; The 16-bit port address is taken in DX.
   OUT DX, AX     ; Write the content of AX to a port of which address is in DX.
```

XLAT: Translate

The translate instruction is used for finding out the codes in case of code conversion problems, using look up table technique.

- Suppose, a hexadecimal key pad having 16 keys from 0 to F is interfaced with 8086 using 8255.
- Whenever a key is pressed, the code of that key (0 to F) is returned in AL.
- For displaying the number corresponding to the pressed key on the 7-segment display device, it is required that the 7-segment code corresponding to the key pressed is found out and sent to the display port.
- This translation from the code of the key pressed to the corresponding 7-segment code is performed using XLAT instruction.



- For this purpose, one is required to prepare a look up table of codes, starting from an offset say 2000H, and store the 7-segment codes for 0 to F at the locations 2000H to 200FH sequentially.
- For executing the XLAT instruction, the code of the pressed key obtained from the keyboard ( i.e. the code to be translated) is moved in AL and the base address of the look up table containing the 7-segment codes is kept in BX.

- After the execution of the XLAT instruction, the 7-segment code corresponding to the pressed key is returned in AL, replacing the key code which was in AL prior to the execution of the XLAT instruction.
- To find out the exact address of the 7-segment code from the base address of look up table, the content of AL is added to BX internally, and the contents of the address pointed to by this new content of BX in DS are transferred to AL.
- The following sequence of instructions perform the task.



Example

```
MOV AX, SEG TABLE   ; Address of the segment containing look-up-table
MOV DS,AX           ; is transferred in DS
MOV AL, CODE        ; Code of the pressed key is transferred in AL
MOV BX, OFFSET TABLE; Offset of the code look-up-table in BX
XLAT                ; Find the equivalent code and store in AL
```

## LEA: Load Effective Address

- The load effective address instruction loads the effective address formed by destination operand into the specified source register. This instruction is more useful for assembly language rather than for machine language.

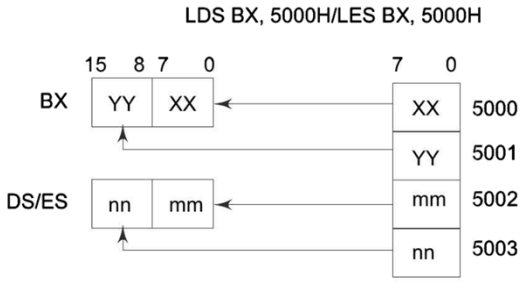
Example

```
LEA BX,ADR      ; Effective address of Label ADR i.e. offset of ADR will be
                  transferred to Reg ; BX.
LEA SI, ADR[Bx]; offset of Label ADR will be added to content of Bx to form effective
                ; address and it will be loaded in SI
```



## LDS/LES: Load Pointer to DS/ES

- This instruction loads the DS or ES register and the specified destination register in the instruction with the content of memory location specified as source in the instruction.




LDS/LES Instruction Execution

LAHF : Load AH from Lower Byte of Flag

This instruction loads the AH register with the lower byte of the flag register.

This command may be used to observe the status of all the condition code flags (except over flow) at a time.



SAHF: Store AH to Lower Byte of Flag Register

- This instruction sets or resets the condition code flags (except overflow) in the lower byte of the flag register depending upon the corresponding bit positions in AH.
- If a bit in AH is 1, the flag corresponding to the bit position is set, else it is reset.

PUSHF: Push Flags to Stack

- The push flag instruction pushes the flag register on to the stack; first the upper byte and then the lower byte is pushed on to it.
- The SP is decremented by 2, for each push operation.

The general operation of this instruction is similar to the PUSH operation.

## POPF: Pop Flags from Stack

- The pop flags instruction loads the flag register completely (both bytes) from the word contents of the memory location currently addressed by SP and SS.
- The SP is incremented by 2 for each pop operation.



Arithmetic Instructions

Arithmetic Instructions

These instructions perform the arithmetic operations, like addition, subtraction, multiplication and division along with the respective ASCII and decimal adjust instructions. The increment and decrement operations also belong to this type of instructions. The 8086/8088 instructions falling under this category are discussed below in significant details. The arithmetic instructions affect all the condition code flags. The operands are either the registers or memory locations or immediate data depending upon the addressing mode.



ADD: Add
This instruction adds an immediate data or contents of a memory location specified in the instruction or a register (source) to the contents of another register (destination) or memory location. The result is in the destination operand. However, both the source and destination operands cannot be memory
operands. That means memory to memory addition is not possible. Also the contents of the segment registers
cannot be added using this instruction. All the condition code flags are affected, depending upon the result.
The examples of this instruction are given along with the corresponding modes.

```
1.ADD AX, 0100H       Immediate
2.ADD AX, BX          Register
3.ADD AX, [SI]        Register indirect
4.ADD AX, [5000H]     Direct
5.ADD [5000H], 0100H  Immediate
6.ADD 0100H           Destination AX (implicit)
```



ADC: Add with Carry
This instruction performs the same operation as ADD instruction, but adds the
carry flag bit (which may be set as a result of the previous calculations) to the result. All the condition code
flags are affected by this instruction.

```
1.ADC 0100H            Immediate (AX implicit)
2.ADC AX, BX           Register
3.ADC AX, [SI]         Register indirect
4.ADC AX, [5000H]      Direct
5.ADC [5000H], 0100H   Immediate
```



INC:Increment
This instruction increases the contents of the specified register or memory location by
1. All the condition code flags are affected except the carry flag CF. This instruction adds 1 to the contents of the operand. Immediate data cannot be operand of this instruction.

```
1. INC    AX       Register
2. INC    [BX]     Register indirect
3. INC    [5000H]  Direct
```



DEC: Decrement
The decrement instruction subtracts 1 from the contents of the specified register
or memory location. All the condition code flags, except the carry flag, are affected depending upon the result. Immediate data cannot be operand of the instruction.

```
1.DEC     AX       Register
2.DEC     [5000H]  Direct
```



SUB:Subtract
The subtract instruction subtracts the source operand from the destination operand and
the result is left in the destination operand. Source operand may be a register, memory location or immediate data and the destination operand may be a register or a memory location, but source and destination operands both must not be memory operands. Destination operand can not be an immediate data. All the condition code flags are affected by this instruction.

```
1.SUB     AX, 0100H      Immediate [destination AX]
2.SUB     AX, BX         Register
3.SUB     AX, [5000H]    Direct
4.SUB     [5000H], 0100  Immediate
```



SBB: Subtract with Borrow
The subtract with borrow instruction subtracts the source operand and
the borrow flag (CF) which may reflect the result of the previous calculations, from the destination operand.
Subtraction with borrow, here means subtracting 1 from the subtraction obtained by SUB, if carry (borrow) flag is set.
The result is stored in the destination operand. Allthe flagsare affected (Condition code) by this instruction.

```
1.SBB    AX, 0100H      Immediate [destination AX]
2.SBB    AX, BX         Register
3.SBB    AX, [5000H]    Direct
4.SBB    [5000H], 0100  Immediate
```

CMP: Compare
This instruction compares the source operand, which may be a register or an im¬
mediate data or a memory location, with a destination operand that may be a register or a memory location. For comparison, it subtracts the source operand from the destination operand but does not store the result anywhere. The flags are affected depending upon the result of the subtraction. If both of the operands are equal, zero flag is set. If the source operand is greater than the destination operand, carry flag is set or else, carry flag is reset.



```
1.CMP BX, 0100H          Immediate
2.CMP AX, 0100H          Immediate
3.CMP [5000H], 0100H     Direct
4.CMP BX, [SI]           Register indirect
5.CMP BX, CX             Register
```

AAA: ASCII Adjust After Addition
The AAA instruction is executed after an ADD instruction that
adds two ASCII coded operands to give a byte of result in AL. The AAA instruction converts the resulting
contents of AL to unpacked decimal digits. After the addition, the AAA instruction examines the lower 4
bits of AL to check whether it contains a valid BCD number in the range 0 to 9. If it is between 0 to 9 and
AF is zero, AAA sets the 4 high order bits of AL to 0. The AH must be cleared before addition. If the lower
digit of AL is between 0 to 9 and AF is set, 06 is added to AL.The upper 4 bits of AL are cleared and AH is
incremented by one. If the value in the lower nibble of AL is greater than 9 then the AL is incremented by 06,
AH is incremented by 1, the AF and CF flags are set to 1, and the higher 4 bits of AL are cleared to 0. The
remaining flags are unaffected. The AH is modified as sum of previous contents (usually 00) and the carry
from the adjustment, as shown in Fig. 2.6. This instruction does not give exact ASCII codes of the sum, but
they can be obtained by adding 3030H to AX.



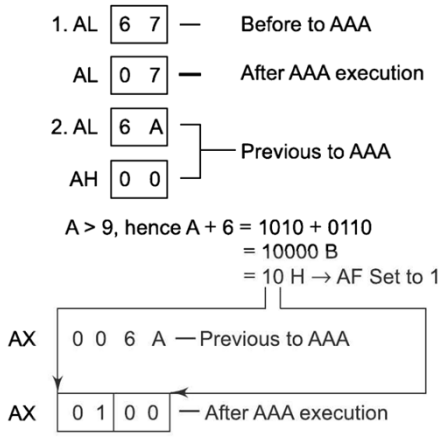



Fig. 2.6
ASCII Adjust after Addition Instruction

AAS: ASCII Adjust AL after Subtraction
AAS instruction corrects the result in AL register after subtracting two unpacked ASCII operands. The result is in unpacked decimal format. If the lower 4 bits of AL register are greater than 9 or if the AF flag is 1, the AL is decremented by 6 and AH register is decre¬ mented by 1, the CF and AF are set to 1. Otherwise, the CF and AF are set to 0, the result needs no correction. As a result, the upper nibble of AL is 00 and the lower nibble may be any number from 0 to 9. The procedure is similar to the AAA instruction except for the subtraction of 06 from AL. AH is modified as difference of the previous contents (usually zero) of AH and the borrow for adjustment.

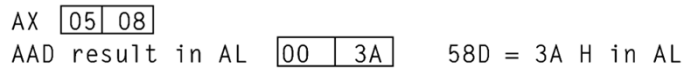


AAM : ASCII Adjust after Multiplication
This instruction, after execution, converts the product available in AL into unpacked BCD format. The AAM—ASCII Adjust After Multiplication—instruction follows a multiplication instruction that multiplies two unpacked BCD operands, i.e. higher nibbles of the multiplication operands should be O. The multiplication of such operands is carried out using MU1instruction. Obviously the result of multiplication is available in AX. The following AAM instruction replaces content of AH by tens of the decimal multiplication and AL by singles of the decimal multiplication.

```
MOV AL, 04  ;  AL ← 04
MOV BL, 09  ;  BL ← 09
MUL BL      ;  AH-AL ← 24H (9 × 4)
AAM         ;  AH ← 03
            ;  AL ← 06
```



AAD: ASCII Adjust before Division
Though the names of these two instructions (AAM and AAD) appear to be similar, there is a lot of difference between their functions. The AAD instruction con¬verts two unpacked BCD digits in AH and AL to the equivalent binary number in AL. This adjustment must be made before dividing the two unpacked BCD digits in AX by an unpacked BCD byte. PF, SF, ZF are modified while AF, CF, OF are undefined, after the execution of the instruction AAD. The example explains the execution of the instruction. In the instruction sequence, this instruction appears before DIV instruction unlike AAM appears after MUL. Let AX contains 0508 unpacked BCD for 58 decimal, and DH contains 02H.




The result of AAD execution will give the hexadecimal number 3A in AL and 00 in AH. Note that 3A is the hexadecimal equivalent of 58 (decimal). Now, instruction DIV DH may be executed. So rather than ASCII adjust for division, it is ASCII adjust before division. All the ASCII adjust instructions are also called as unpacked BCD arithmetic instructions. Now, we will consider the two instructions related to packed BCD arithmetic.



DAA: Decimal Adjust Accumulator
This instruction is used to convert the result of the addition of two packed BCD numbers to a valid BCD number. The result has to be only in AL. If the lower nibble is greater than 9, after addition or if AF is set, it will add 06 to the lower nibble in AL. After adding 06 in the lower nibble of AL, if the upper nibble of AL is greater than 9 or if carry flag is set, DAA instruction adds 60H to AL. The examples given below explain the instruction.

```
(i) AL = 53       CL = 29
    ADD AL, CL    ; AL ← (AL) + (CL)
                  ; AL ← 53 + 29
                  ; AL ← 7C
    DAA           ; AL ← 7C + 06 (as C>9)
                  ; AL ← 82
```

```
(ii) AL = 73      CL = 29
     ADD AL, CL   ; AL ← AL + CL
                  ; AL ← 73 + 29
                  ; AL ← 9C
     DAA          ; AL ← 02 and CF = 1
```

```
  AL = 7 3
    +
CL = 2 9
     9 C
   + 6
     A 2
   + 6 0
  CF = 1  0 2  in AL
```

The instruction DAA affects AF, CF, PF, and ZF flags. The OF is undefined.



DAS: Decimal Adjust after Subtraction
This instruction converts the result of subtraction of two packed BCD numbers to a valid BCD number. The subtraction has to be in AL only. If the lower nibble of AL is greater than 9, this instruction will subtract 06 from lower nibble of AL. If the result of subtraction sets the carry flag or if upper nibble is greater than 9, it subtracts 60H from AL. This instruction modifies the AF, CF, SF, PF and ZF flags. The OF is undefined after DAS instruction.

```
(i) AL = 75       BH = 46
    SUB AL, BH    ; AL ← 2 F = (AL) - (BH)
                  ; AF = 1
    DAS           ; AL ← 2 9 (as F > 9, F - 6 = 9)
(ii) AL = 38      CH = 6 1
    SUB AL, CH    ; AL ← D 7  CF = 1 (borrow)
    DAS           ; AL ← 7 7  (as D > 9, D - 6 = 7)
                  ; CF = 1 (borrow)
```

DAA and DAS instructions are also called packed BCD arithmetic instructions.



NEG: Negate
The negate instruction forms 2's complement of the specified destination in the
instruction. For obtaining 2's complement, it subtracts the contents of destination from zero. The result is
stored back in the destination operand which may be a register or a memory location. If OF is set, it indicates that the operation could not be completed successfully. This instruction affects all the condition code flags.

MUL: Unsigned Multiplication Byte or Word
This instruction multiplies an unsigned byte or word by the contents of AL. The unsigned byte or word may be in any one of the general purpose registers or memory locations. The most significant word of the result is stored in DX, while the least significant word of the result is stored in AX. All the flags are modified depending upon the result. The example instructions are as shown. Immediate operand is not allowed in this instruction. If the most significant byte or word of the result is '0' CF and OF both will be set.



```
1. MUL BH             ; (AX) ← (AL) × (BH)
2. MUL CX             ; (DX) (AX) ← (AX) × (CX)
3. MUL WORD PTR [SI]  ; (DX) (AX) ← (AX) × ([SI])
```

IMUL: Signed Multiplication
This instruction multiplies a signed byte in source operand by a signed
byte in AL or a signed word in source operand by a signed word in AX. The source can be a general purpose register, memory operand, index register or base register, but it cannot be an immediate data. In case of 32-bit results, the higher order word (MSW) is stored in DX and the lower order word is stored in AX. The AF, PF, SF, and ZF flags are undefined after IMUL. If AH and DX contain parts of 16 and 32-bit result respectively, CF and OF both will be set. The AL and AX are the implicit operands in case of 8 bits and 16 bits multipli¬ cations respectively. The unused higher bits of the result are filled by sign bit and CF, AF are cleared.



```
1.IMUL    BH
2.IMUL    CX
3.IMUL    [SI]
```

CBW: Convert Signed Byte to Word
This instruction converts a signed byte to a signed word. In
other words, it copies the sign bit of a byte to be converted to all the bits in the higher byte of the result word.
The byte to be converted must be in AL. The result will be in AX. It does not affect any flag.
CWD: Convert Signed Word to Double Word
This instruction copies the sign bit of AX to all the bits of the DX register. This operation is to be done before signed division. It does not affect any flag.



DIV: Unsigned Division
This instruction performs unsigned division. It divides an unsigned word or
double word by a 16-bit or 8-bit operand. The dividend must be in AX for 16-bit operation and divisor may be specified using any one of the addressing modes except immediate. The result will be in AL (quotient) while AH will contain the remainder. If the result is too big to fit in AL, type 0 (divide by zero) and an interrupt is generated. In case of a double word dividend (32-bit), the higher word should be in DX and lower word should be in AX. The divisor may be specified as already explained. The quotient and the remainder, in this case, will be in AX and DX respectively. This instruction does not affect any flag.

IDIV: Signed Division
This instruction performs the same operation as the DIV instruction, but with
signed operands. The results are stored similarly as in case of DIV instruction in both cases of word and double word divisions. The results will also be signed numbers. The operands are also specified in the same way as DIV instruction. Divide by 0 interrupt is generated, if the result is too big to fit in AX (16-bit dividend operation) or AX and DX (32-bit dividend operation). All the flags are undefined after IDIV instruction.



# Logical Instructions

Logical Instructions

- These type of instructions are used for carrying out the bit by bit shift, rotate, or basic logical operations.
- All the condition code flags are affected depending upon the result.
- Basic logical operations available with 8086 instruction set are AND, OR, NOT, and XOR.



AND: Logical AND
This instruction bit by bit ANDs the source operand that may be an immediate,
a register or a memory location to the destination operand that may be a register or a memory location. The result is stored in the destination operand. At least one of the operands should be a register or a memory operand. Both the operands cannot be memory locations or immediate operands. An immediate operand can¬ not be a destination operand.

```
1.AND AX, 0008H
2.AND AX, BX
3.AND AX, [5000H]
4.AND [5000H], DX
```

If the content of AX is 3F0FH, the first example instruction will carry out the operation as given below. The result 3F9FH will be stored in the AX register.

```
  0 0 1 1     1 1 1 1     0 0 0 0     1 1 1 1     = 3F0F H [AX]
  ↓ ↓ ↓ ↓     ↓ ↓ ↓ ↓     ↓ ↓ ↓ ↓     ↓ ↓ ↓ ↓       AND
  0 0 0 0     0 0 0 0     0 0 0 0     1 0 0 0     = 0008 H
 ---------------------------------------------------------
  0 0 0 0     0 0 0 0     0 0 0 0     1 0 0 0     = 0008 H [AX]
```

The result 0008H will be in AX.



OR: Logical OR
The OR instruction carries out the OR operation in the same way as described in case
of the AND operation. The limitations on source and destination operands are also the same as in case of AND operation.

```
1.OR AX, 0098H
2.OR AX, BX
3.OR AX, [5000H]
4.OR [5000H], 0008H
```

The contents of AX are say 3F0FH, then the first example instruction will be carried out as given below.

```
  0 0 1 1     1 1 1 1     0 0 0 0     1 1 1 1     = 3F0F H
  ↓ ↓ ↓ ↓     ↓ ↓ ↓ ↓     ↓ ↓ ↓ ↓     ↓ ↓ ↓ ↓       OR
  0 0 0 0     0 0 0 0     1 0 0 1     1 0 0 0     = 0098 H
-------------------------------------------------------------
  0 0 1 1     1 1 1 1     1 0 0 1     1 1 1 1     = 3F9F H
```

Thus the result 3F9FH will be stored in the AX register.



NOT: Logical Invert
The NOT instruction complements (inverts) the contents of an operand register
or a memory location, bit by bit.

```
NOT AX
NOT [5000H]
```

If the content of AX is 200FH, the first example instruction will be executed as shown.

```
   AX      = 0 0 1 0     0 0 0 0     0 0 0 0     1 1 1 1
invert       ↓ ↓ ↓ ↓     ↓ ↓ ↓ ↓     ↓ ↓ ↓ ↓     ↓ ↓ ↓ ↓
             1 1 0 1     1 1 1 1     1 1 1 1     0 0 0 0
```

**Result**

```
in AX =         D             F             F             0
```

The result DFF0H will be stored in the destination register AX.



XOR: Logical Exclusive OR
The XOR operation is again carried out in a similar way to the AND and
OR operation. The constraints on the operands are also similar. The XOR operation gives a high output, when the 2 input bits are dissimilar. Otherwise, the output is zero.

```
1.XOR   AX, 0098H
2.XOR   AX, BX
3.XOR   AX, [5000H]
```

If the content of AX is 3F0FH, then the first example instruction will be executed as explained. The result 3F97H will be stored in AX.

```
 AX = 3F0FH =      0 0 1 1     1 1 1 1     0 0 0 0     1 1 1 1
        XOR        ↓ ↓ ↓ ↓     ↓ ↓ ↓ ↓     ↓ ↓ ↓ ↓     ↓ ↓ ↓ ↓
      0098H =      0 0 0 0     0 0 0 0     1 0 0 1     1 0 0 0
AX = Result =      0 0 1 1     1 1 1 1     1 0 0 1     0 1 1 1
           = 3F97H
```



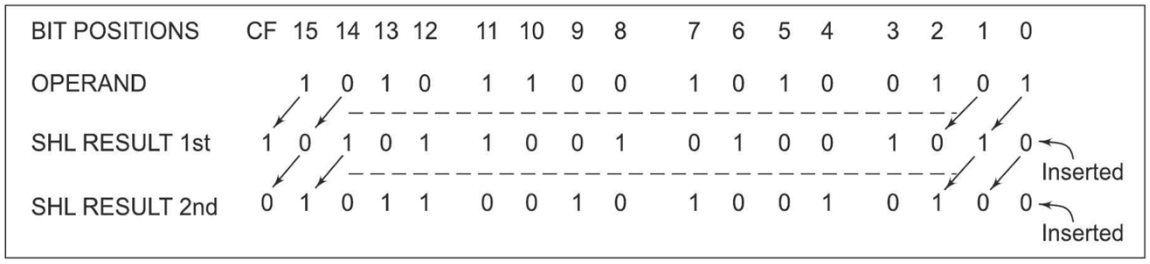
TEST: Logical Compare Instruction

- The TEST instruction performs a bit by bit logical AND operation on the two operands.
- Each bit of the result is then set to 1, if the corresponding bits of both operands are 1, else the result bit is reset to 0. The result of this ANDing operation is not available for further use, but flags are affected.
- The affected flags are OF, CF, SF, ZF and PF.
- The operands may be registers, memory or immediate data.

```
1.TEST    AX, BX
2.TEST    [0500], 06H
3.TEST    [BX] [DI], CX
```



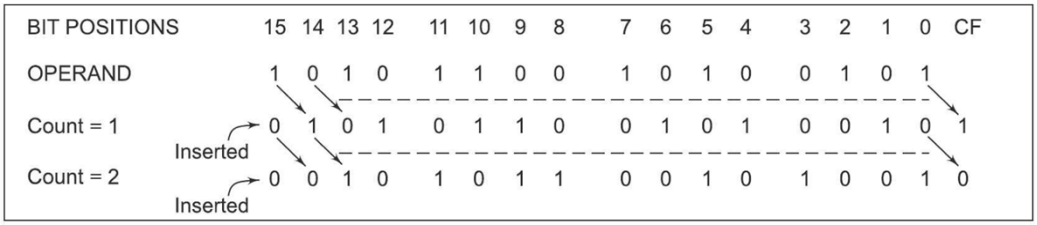
SHL/SAL: Shift Logical/Arithmetic Left
These instructions shift the operand word or byte bit by bit to the left and insert zeros in the newly introduced least significant bits. In case of all the SHIFT and ROTATE instructions, the count is either 1 or specified by register CL. The operand may reside in a register or a memory location but cannot be an immediate data. All flags are affected depending upon the result. Figure 2.7 explains the
execution of this instruction. It is to be noted here that the shift operation is through carry flag.




Execution of SHL/SAL Instruction



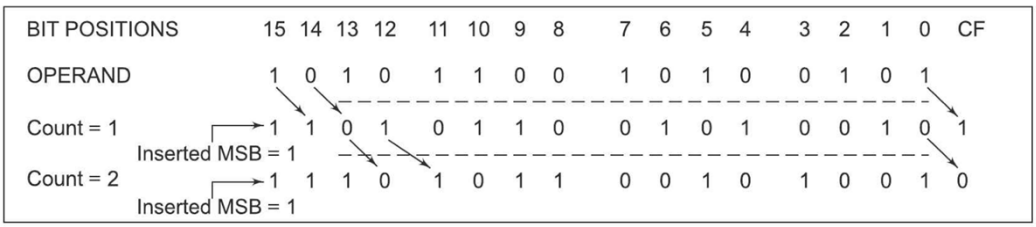
SHR: Shift Logical Right
This instruction performs bit-wise right shifts on the operand word or byte
that may reside in a register or a memory location, by the specified count in the instruction and inserts zeros
in the shifted positions. The result is stored in the destination operand. Figure 2.8 explains execution of this instruction. This instruction shifts the operand through the carry flag.




Execution of SHR Instruction



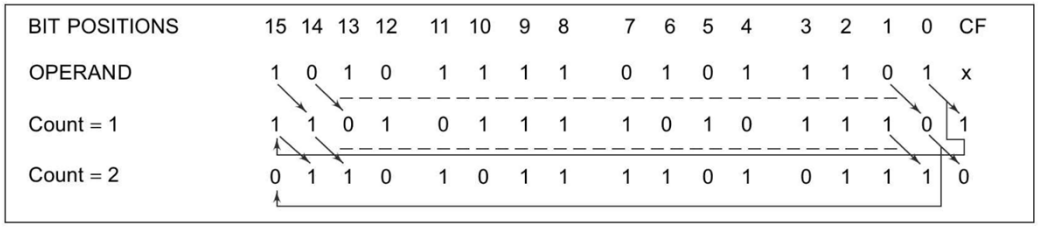
SAR: Shift Arithmetic Right
This instruction performs right shifts on the operand word or byte, that
may be a register or a memory location by the specified count in the instruction. It inserts the most significant bit of the operand in the newly inserted positions. The result is stored in the destination operand. Figure 2.9 explains execution of the instruction. All the condition code flags are affected. This shift operation shifts the operand through the carry flag.




Execution of SAR Instruction
Immediate operand is not allowed in any of the shift instructions.



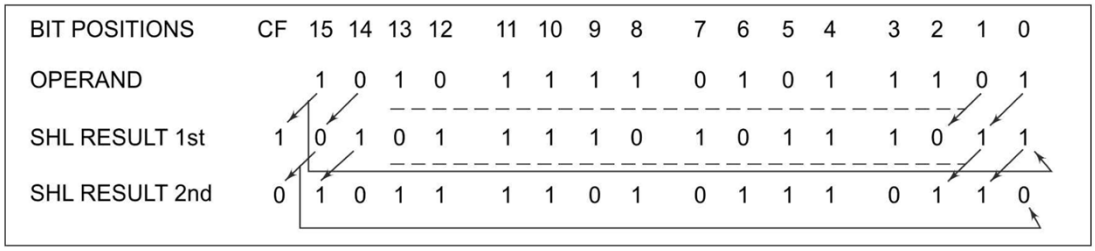
ROR: Rotate Right without Carry
This instruction rotates the contents of the destination operand
to the right (bit-wise) either by one or by the count specified in CL, excluding carry. The least significant bit is pushed into the carry flag and simultaneously it is transferred into the most significant bit position at each operation. The remaining bits are shifted right by the specified positions. The PF, SF, and ZF flags are left unchanged by the rotate operation. The operand may be a register or a memory location but it cannot be an immediate operand. Figure 2.10 explains the operation. The destination operand may be a register (except a segment register) or a memory location.




Execution of ROR Instruction



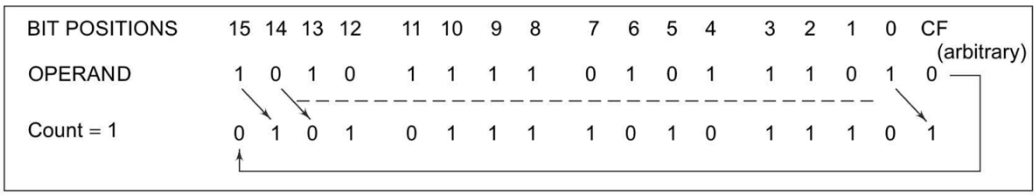
ROL: Rotate Left without Carry
This instruction rotates the content of the destination operand to
the left by the specified count (bit-wise) excluding carry. The most significant bit is pushed into the carry flag as well as the least significant bit position at each operation. The remaining bits are shifted left subsequently by the specified count positions. The PF, SF, and ZF flags are left unchanged in this rotate operation. The operand may be a register or a memory location. Figure 2.11 explains the operation.




Execution of ROL Instruction



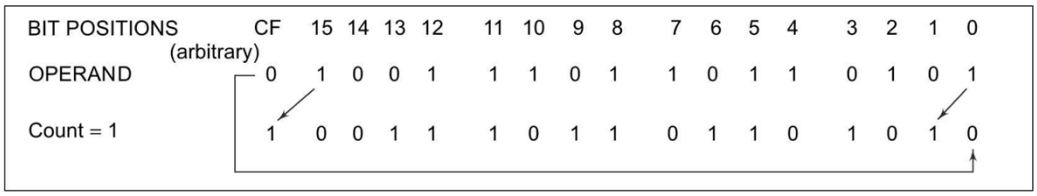
RCR: Rotate Right through Carry
This instruction rotates the contents (bit-wise) of the destination
operand right by the specified count through carry flag (CF). For each operation, the carry flag is pushed into the MSB of the operand, and the LSB is pushed into carry flag. The remaining bits are shifted right by the specified count positions. The SF, PF, ZF are left unchanged. The operand may be a register or a memory location. Figure 2.12 explains the operation.




Execution of RCR Instruction



RCL: Rotate Left through Carry
This instruction rotates (bit-wise) the contents of the destination
operand left by the specified count through the carry flag (CF). For each operation, the carry flag is pushed
into LSB and the MSB of the operand is pushed into carry flag. The remaining bits are shifted left by the
specified positions. The SF, PF, ZF are left unchanged.
The operand may be a register or a memory location.
Figure 2.13 explains the operation.




Execution of RCL Instruction
The count for rotation or shifting is either 1 or is specified using register CL, in case of all the shift and rotate instructions.



String Manipulation Instructions

- A series of data bytes or words available in memory at consecutive locations, to be referred to collectively or individually, are called as byte strings or word strings.
- For example, a string of characters may be located in consecutive memory locations, where each character may be represented by its ASCII equivalent.
- For referring to a string, two parameters are required, (a) starting or end address of the string and (b) length of the string.
- The length of a string is usually stored as count in the CX register. In case of 8085, similar structures can be set up by the pointer and counter arrangements which may be modified at each iteration, till the required condition for proceeding further is satisfied.
- On the other hand, the 8086 supports a set of more powerful instructions for string manipulations.
- The incrementing or decrementing of the pointer, in case of 8086 string instructions, depends upon the Direction Flag (DF) status.
- If it is a byte string operation, the index registers are updated by one. On the other hand, if it is a word string operation, the index registers are updated by two.
- The counter in both the cases, is decremented by one.

REP: Repeat Instruction Prefix

- This instruction is used as a prefix to other instructions.
- The instruction to which the REP prefix is provided, is executed repeatedly until the CX register becomes zero (at each iteration CX is automatically decremented by one).
- When CX becomes zero, the execution proceeds to the next instruction in sequence.
- There are two more options of the REP instruction. The first is REPE/REPZ, i.e. repeat operation while equal/zero.
- The second is REPNE/REPNZ allows for repeating the operation while not equal/not zero.
- These options are used for CMPS, SCAS instructions only, as instruction prefixes.



MOVSB/MOVSW: Move String Byte or String Word

- Suppose a string of bytes stored in a set of consecutive memory locations is to be moved to another set of destination locations.
- The starting byte of the source string is located in the memory location whose address may be computed using SI (Source Index) and DS (Data Segment) contents.
- The starting address of the destination locations where this string has to be relocated is given by DI (Destination Index) and ES (Extra Segment) contents.
- The starting address of the source string is 10H*DS+[SI], while the starting address of the destination string is 10H*ES+[DI], The MOVSB/MOVSW instruction thus, moves a string of bytes/words pointed to by DS: SI pair (source) to the memory location pointed to by ES: DI pair (destination).
- The REP instruction prefix is used with MOVS instruction to repeat it by a value given in the counter (CX).
- The length of the byte string or word string must be stored in CX register. No flags are affected by this instruction.

After the MOVS instruction is executed once, the index registers are automatically updated and CX is decre¬
mented. The incrementing or decrementing of the pointers, i.e. SI and DI depend upon the direction flag DF. If DF
is 0, the index registers are incremented, otherwise, they are decremented, in case of all the string manipulation
instructions. The following string of instructions explain the execution of the MOVS instruction.



Example

```
MOV AX, 5000H    ; Source segment address is 5000h
MOV DS, AX       ; Load it to DS
MOV AX, 6000H    ; Destination segment address is 6000h
MOV ES, AX       ; Load it to ES
MOV CX, 0FFH     ; Move length of the string to counter register CX
MOV SI, 1000H    ; Source index address 1000H is moved to SI
MOV DI, 2000H    ; Destination index address 2000H is moved to DI
CLD              ; Clear DF, i.e. set autoincrement mode
REP MOVSB        ; Move 0FFH string bytes from source address to destination
```

CMPS: Compare String Byte or String Word

- The CMPS instruction can be used to compare two strings of bytes or words.
- The length of the string must be stored in the register CX. If both the byte or word strings are equal, zero flag is set.
- The flags are affected in the same way as CMP instruction.
- The DS:SI and ES:DI point to the two strings.
- The REP instruction prefix is used to repeat the operation till CX(counter) becomes zero or the condition specified by the REP prefix is false.

The following string of instructions explain the instruction.

- The comparison of the string starts from initial byte or word of the string, after each comparison the index registers are updated depending upon the direction flag and the counter is decremented.
- This byte by byte or word by word comparison continues till a mismatch is found.
- When, a mismatch is found, the carry and zero flags are modified appropriately and the execution proceeds further.



```
MOV AX, SEG1           ; Segment address of STRING1, i.e. SEG1 is moved
                         to AX
MOV DS, AX             ; Load it to DS
MOV AX, SEG2           ; Segment address of STRING2, i.e. SEG2 is moved
                         to AX
MOV ES, AX             ; Load it to ES
MOV SI, OFFSET STRING1 ; Offset of STRING1 is moved to SI
MOV DI, OFFSET STRING2 ; Offset of STRING2 is moved to DI
MOV CX, 010H           ; Length of the string is moved to CX
CLD                    ; Clear DF, i.e. set autoincrement mode
REPE CMPSW             ; Compare 010H words of STRING1 and
                       ; STRING2, while they are equal, If a mismatch is found,
                       ; modify the flags and proceed with further execution
```

If both strings are completely equal, i.e. CX becomes zero, the ZF is set, otherwise, ZF is reset.

SCAS: Scan String Byte or String Word
This instruction scans a string of bytes or words for an
operand byte or word specified in the register AL or AX. The string is pointed to by ES:DI register pair. The
length of the string is stored in CX. The DF controls the mode for scanning of the string, as stated in case
of MOVSB instruction. Whenever a match to the specified operand, is found in the string, execution stops
and the zero flag is set. If no match is found, the zero flag is reset. The REPNE prefix is used with the SCAS
instruction. The pointers and counters are updated automatically, till a match is found. The following string
of instructions elaborates the use of SCAS instruction.



Example

```
  MOV AX,SEG    ; Segment address of the string, i.e. SEG is moved to AX
  MOV ES,AX     ; Load it to ES
  MOV DI,OFFSET ; String offset, i.e. OFFSET is moved to DI
  MOV CX,010H   ; Length of the string is moved to CX
  MOV AX,WORD   ; The word to be scanned for, i.e. WORD is in AL
  CLD           ; Clear DF
REPNE SCASW     ; Scan the 010H bytes of the string, till a match to
                ; WORD is found
```

This string of instructions finds out, if it contains WORD. If the WORD is found in the word string, before CX becomes zero, the ZF is set, otherwise the ZF is reset. The scanning will continue till a match is found. Once a match is found the execution of the programme proceeds further.

LODS: Load String Byte or String Word
The LODS instruction loads the AL/AX register by the content ofa string pointed to by DS:SI register pair. The SI is modified automatically depending upon DF. The DF plays exactly the same role as in case of MOVSB/MOVSW instruction. If it is a byte transfer(LODSB),
the SI is modified by one and if it is a word transfer(LODSW), the SI is modified by two. No other flags are
affected by this instruction.



STOS: Store String Byte or String Word
The STOS instruction stores the AL/AX register con¬ tents to a location in the string pointed by ES: DI register pair. The DI is modified accordingly. No flags are affected by this instruction.

The direction flag controls the string instruction execution. The source index SI and destination index DI are modified after each iteration automatically. If DF = 1, then the execution follows autodecrement mode. In this mode, SI and DI are decremented automatically after each iteration (by 1 or 2 depending upon byte or word operations). Hence, in autodecrementing mode, the strings are referred to by their ending addresses. If DF = 0, then the execution follows autoincrement mode. In this mode, SI and DI are incremented automati¬ cally (by 1 or 2 depending upon byte or word operation) after each iteration, hence the strings, in this case, are referred to by their starting addresses. Chapter 3 on assembly language programming explains the use of some of these instructions in assembly language programs.



Control Transfer or Branching Instructions

## Control Transfer or Branching Instructions

The control transfer instructions transfer the flow of execution of the program to a new address specified in the instruction directly or indirectly. When this type of instruction is executed, the CS and IP registers get loaded with new values of CS and IP corresponding to the location where the flow of execution is going to be transferred. Depending upon the addressing modes specified in Chapter 1, the CS may or may not be modi¬ fied. This type of instructions are classified in two types:



Unconditional Control Transfer (Branch) Instructions
In case of unconditional control transfer instructions, the execution control is transferred to the specified location independent of any status or condi¬ tion. The CS and IP are unconditionally modified to the new CS and IP.

Conditional Control Transfer (Branch) Instructions
In the conditional control transfer instruc¬
tions, the control is transferred to the specified location provided the result of the previous operation satisfies a particular condition, otherwise, the execution continues in normal flow sequence. The results of the previ¬
ous operations are replicated by condition code flags. In other words, using this type of instruction the control will be transferred to a particular specified location, if a particular flag satisfies the condition.



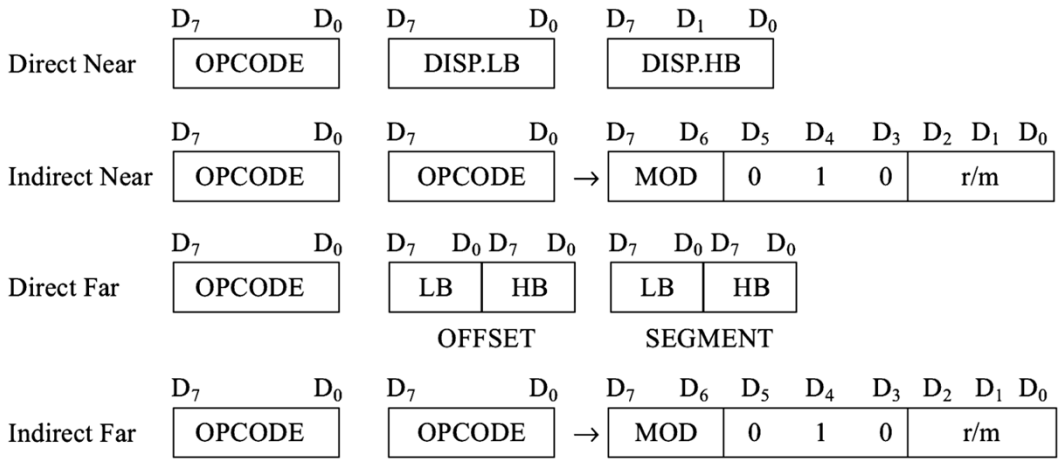
Unconditional Branch Instructions

Unconditional Branch Instructions

CALL: Unconditional Call

This instruction is used to call a subroutine from a main program. In case of assembly language programming, the term procedure is used interchangeably with subroutine. The ad¬ dress of the procedure may be specified directly or indirectly depending upon the addressing mode. There are again two types of procedures depending upon whether it is available in the same segment (Near CALL, i.e. ±32K displacement) or in another segment (FAR CALL, i.e. anywhere outside the segment). The modes for them are called as intrasegment and intersegment addressing modes respectively. This instruction comes un¬ der unconditional branch instructions and can be described as shown with the coding formats. On execution, this instruction stores the incremented IP (i.e. address of the next instruction) and CS onto the stack and loads the CS and IP registers, respectively, with the segment and offset addresses of the procedure to be called. In case of NEAR CALL it pushes only IP register and in case of FAR CALL it pushes IP and CS both onto the stack. The NEAR and FAR CALLS are discriminated using opcode.





RET: Return from the Procedure
At each CALL instruction, the IP and CS of the next instruction
is pushed onto stack, before the control is transferred to the procedure. At the end of the procedure, the RET instruction must be executed. When it is executed, the previously stored content of IP and CS along with flags are retrieved into the CS, IP and flag registers from the stack and the execution of the main program continues further. The procedure may be a near or a far procedure. In case of a FAR procedure, the current contents of SP points to IP and CS at the time of return. While in case of a NEAR procedure, it points to only IP. Depend¬ ing upon the type of procedure and the SP contents, the RET instruction is of four types.

1.
Return within segment
2.
Return within segment adding 16-bit immediate displacement to the SP contents.
3.
Return intersegment
4.
Return intersegment adding 16-bit immediate displacement to the SP contents.



INT N: Interrupt Type N
In the interrupt structure of 8086/8088, 256 interrupts are defined corre¬
sponding to the types from 00H to FFH. When an INT N instruction is executed, the TYPE byte N is multi¬
plied by 4 and the contents of IP and CS of the interrupt service routine will be taken from the hexadecimal multiplication (N'4) as offset address and 0000 as segment address. In other words, the multiplication of type N by 4 (offset) points to a memory block in 0000 segment, which contains the IP and CS values of the interrupt service routine. For the execution of this instruction, the IF must be enabled.

Example

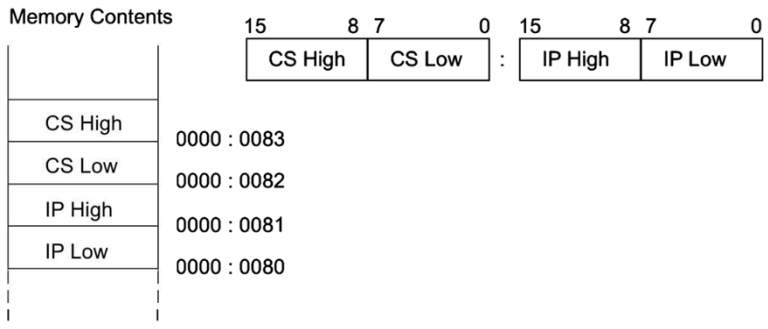
Thus the instruction INT 20H will find out the address of the interrupt service routine as follows:

```
INT      20H
Type* 4 = 20 * 4 = 80H
```

Pointer to IP and CS of the ISR is 0000 : 0080 H



the arrangement of CS and IP addresses of the ISR in the interrupt vector table.




Contents of IVT

INTO: Interrupt on Overflow
This command is executed, when the overflow flag OF is set. The
new contents of IP and CS are taken from the address 0000:0010 as explained in INT type instruction. This is equivalent to a Type 4 interrupt instruction



JMP: Unconditional Jump
This instruction unconditionally transfers the control of execution to the
specified address using an 8-bit or 16-bit displacement (intrasegment relative, short or long) or CS: IP (in¬ tersegment direct far). No flags are affected by this instruction. Corresponding to the methods of specifying jump addresses, the JUMP instruction may have the following three formats. For other JMP types the reader may refer to the following datasheet.

| | | | | |
|---|---|---|---|---|
| JUMP | DISP 8-bit | | | Intrasegment, relative, short jump |
| JUMP | DISP.16-bit (LB) | DISP.16-bit (HB) | | Intrasegment, relative, short jump |
| JUMP | IP (LB) | IP (HB) | CS (LB) | S (HB) Intrasegment, direct, far jump |



IRET: Return from ISR
When an interrupt service routine is to be called, before transferring control to it,
the IP,CSand flag registerare stored on to the stack to indicate the location from where the execution is to be con¬
tinued,after the ISR is executed. So,at the end ofeach ISR, when IRET is executed, the valuesof IP,CS and flags
are retrieved from the stack to continue the execution of the main program. The stack is modified accordingly.

## LOOP: Loop Unconditionally

- This instruction executes the part of the program from the label or address specified in the instruction up to the loop instruction, CX number of times.
- The following sequence explains the execution.
- At each iteration, CX is decremented automatically.
- In other words, this instruction implements DECREMENT COUNTER and JUMP IF NOT ZERO structure.



Example

```
          MOV   CX, 0005   ; Number of times in CX
          MOV   BX, 0FF7H  ; Data to BX
Label :   MOV   AX, CODE1
          OR    BX, AX
          AND   DX, AX
          Loop  Label
```

- The execution proceeds in sequence, after the loop is executed, CX number of times.
- If CX is already 00H, the execution continues sequentially. No flags are affected by this instruction.

## Conditional Branch Instructions



## Conditional Branch Instructions

- When these instructions are executed, execution control is transferred to the address specified relatively in the instruction, provided the condition implicit in the opcode is satisfied.
- If not the execution continues sequentially.
- The conditions, here, means the status of condition code flags.
- These type of instructions do not affect any flag.

- The address has to be specified in the instruction relatively in terms of displacement which must lie within -80H to 7FH (or -128 to 127) bytes from the address of the branch instruction.
- In other words, only short jumps can be implemented using conditional branch instructions.
- A label may represent the displacement, if it lies within the above specified range.
- The different 8086/8088 conditional branch instructions and their operations are listed in Table



Conditional Branch Instructions

| | *Mnemonic* | *Displacement* | *Operation* |
|---|---|---|---|
| 1. | JZ/JE | Label | Transfer execution control to address 'Label', if ZF=1. |
| 2. | JNZ/JNE | Label | Transfer execution control to address 'Label', if ZF=0. |
| 3. | JS | Label | Transfer execution control to address 'Label', if SF=1. |
| 4. | JNS | Label | Transfer execution control to address 'Label', if SF=0. |
| 5. | JO | Label | Transfer execution control to address 'Label', if OF=1. |
| 6. | JNO | Label | Transfer execution control to address 'Label', if OF=0. |
| 7. | JP/JPE | Label | Transfer execution control to address 'Label', if PF=1. |
| 8. | JNP | Label | Transfer execution control to address 'Label', if PF=0. |
| 9. | JB/JNAE/JC | Label | Transfer execution control to address 'Label', if CF=1. |
| 10. | JNB/JAE/JNC | Label | Transfer execution control to address 'Label', if CF=0. |
| 11. | JBE/JNA | Label | Transfer execution control to address 'Label', if CF=1 or ZF=1. |
| 12. | JNBE/JA | Label | Transfer execution control to address 'Label', if CF=0 or ZF=0. |
| 13. | JL/JNGE | Label | Transfer execution control to address 'Label', if neither SF=1 nor OF=1. |
| 14. | JNL/JGE | Label | Transfer execution control to address 'Label', if neither SF=0 nor OF=0. |
| 15. | JLE/JNC | Label | Transfer execution control to address 'Label', if ZF=1 or neither SF nor OF is 1. |
| 16. | JNLE/JE | Label | Transfer execution control to address 'Label', if ZF=0 or at least any one of SF and OF is 1(Both SF and OF are not 0). |

- While the remaining instructions can be used for unsigned binary operations, the last four instructions are used in case of decisions based on signed binary number operations.
- The terms above and below are generally used for unsigned numbers, while the terms less and greater are used for signed numbers.
- A conditional jump instruction that does not check status flags for condition testing, is given as follows:

```
JCXZ    'Label'   Transfer execution control
                  to address 'Label', if CX=0.
```



- The conditional LOOP instructions are given in Table with their meanings.
- These instructions may be used for implementing structures like DO_WHILE, REPEAT_UNTIL, etc.

Table 2.4
Conditional Loop Instructions

| *Mnemonic* | *Displacement* | *Operation* |
|---|---|---|
| LOOPZ/LOOPE (Loop while ZF = 1; equal) | Label | Loop through a sequence of instructions from 'Label' while ZF=1 and CX ¹ 0. |
| LOOPNZ/LOOPENE (Loop while ZF = 0; not equal) | Label | Loop through a sequence of instructions from 'Label' while ZF=0 and CX ¹ 0. |

- These instructions will be clear with programming practice.



## Flag Manipulation and Processor Control Instructions

**Flag Manipulation and Processor Control Instructions**

- These instructions control the functioning of the available hardware inside the processor chip.
- These are categorized into two types;
  - (a) flag manipulation instructions and
  - (b) machine control instructions.
- The flag manipulation instructions directly modify some of the flags of 8086.
- The machine control instructions control the bus usage and execution.
- The flag manipulation instructions and their functions are listed in Table



Table
Flag Manipulation Instructions

| | | |
|---|---|---|
| CLC | - | Clear carry flag |
| CMC | - | Complement carry flag |
| STC | - | Set carry flag |
| CLD | - | Clear direction flag |
| STD | - | Set direction flag |
| CLI | - | Clear interrupt flag |
| STI | - | Set interrupt flag |

These instructions modify the Carry (CF), Direction (DF) and Interrupt (IF) flags directly. The DF and IF, which may be modified using the flag manipulation instructions, further control the processor operation; like interrupt responses and autoincrement or autodecrement modes. Thus, the respective instructions may also be called machine or processor control instructions. The other flags can be modified using POPF and SAHF instructions, which are termed as data transfer instructions, in this text. No direct instructions, are available
for modifying the status flags except carry flag.
The machine control instructions supported by 8086 and 8088 are listed in Table 2.6 along with their func¬ tions. They do not require any operand.



Table
Machine Control Instructions

| | | |
|---|---|---|
| WAIT | - | Wait for Test input pin to go low |
| HLT | - | Halt the processor |
| NOP | - | No operation |
| ESC | - | Escape to external device like NDP (numeric co-processor) |
| LOCK | - | Bus lock instruction prefix. |

after executing the HLT instruction, the processor enters the halt state. The two ways to pull it out of the halt state are to reset the processor or to interrupt it. When NOP instruction is executed, the processor does not perform any operation till 4 clock cycles, except for incrementing the IP by one. It then continues with further execution after 4 clock cycles. ESC instruction when executed, frees the bus for an external master like a coprocessor or peripheral devices. The LOCK prefix may appear with another instruction. When it is executed, the bus access is not allowed for another master till the lock pre¬ fixed instruction is executed completely. This instruction is used in case of programming for multiprocessor systems. The WAIT instruction when executed, holds the operation of processor with the current status till the logic level on the ~~TEST~~ pin goes low. The processor goes on inserting WAIT states in the instruction cycle, till the ~~TEST~~ pin goes low. Once the ~~TEST~~ pin goes low, it continues further execution.



ASSEMBLER DIRECTIVES AND OPERATORS

ASSEMBLER DIRECTIVES AND OPERATORS

The main advantage of machine language programming is that the memory control is directly in the hands of the programmer enabling him to manage the memory of the system more efficiently. However, there are more disadvantages. The programming, coding and resource management techniques are tedious. As the program¬ mer has to consider all these functions, the chances of human errors are more. To understand the programs one has to have a thorough technical knowledge of the processor architecture and instruction set.



The assembly language programming is simpler as compared to the machine language programming. The instruction mnemonics are directly written in the assembly language programs. The programs are now more readable than that of machine language programs. The advantage that assembly language has over machine language is that now the address values and the constants can be identified by labels. If the labels are clear then certainly the program will become more understandable, and each time the pro¬ grammer will not have to remember the different constants and the addresses at which they are stored, throughout the programs. Due to this facility, the tedious byte handling and manipulations are got rid of. Similarly, now different logical segments and routines may be assigned with the labels rather than the different addresses. The memory control feature of machine language programming is left unchanged by providing storage define facilities in assembly language programming. The documentation facility which was not possible with machine language programming is now available in assembly language. Readers will get a better glimpse of the different features of assembly language, when we discuss assembly lan¬ guage programming in the next chapter.

An assembler is a program used to convert an assembly language program into the equivalent machine code modules which may further be converted to executable codes. It decides the address of each label and substitutes the values for each of the constants and variables. It then forms the machine code for the mnemon¬ics and data in the assembly language program. While doing these things, the assembler may find out syntax errors.The logical errors or other programming errors are not found out by the assembler. For completing all these tasks, an assembler needs some hints from the programmer, i.e. the required storage for a particular constant or a variable, logical names of the segments, types of the different routines and modules, end of file, etc. These types of hints are given to the assembler using some predefined alphabetical strings called assembler directives, which help the assembler to correctly understand the assembly language programs to prepare the codes.



Another type of hint which helps the assembler to assign a particular constant with a label or initialise particular memory locations or labels with constants is an operator. In fact, the operators perform the arith¬metic and logical tasks unlike directives that just direct the assembler to correctly interpret the program to code it appropriately. The following directives are commonly used in the assembly language programming practice using Microsoft Macro Assembler or Turbo Assembler. The directives and operators are discussed here but their meanings and uses will be more clear in Chapter 3 on assembly language programming techniques.

DB: Define Byte
The DB directive is used to reserve byte or bytes of memory locations in the available
memory. While preparing the EXE file, this directive directs the assembler to allocate the specified number of memory bytes to the said data type that may be a constant, variable, string, etc. Another option of this directive
also initialises the reserved memory bytes with the ASCII codes of the characters specified as a string. The fol¬ lowing examples show how the DB directive is used for different purposes.



Example

```
RANKS    DB 01H, 02H, 03H, 04H
```

This statement directs the assembler to reserve four memory locations for a list named RANKS and initialise them with the above specified four values.

```
MESSAGE DB 'GOOD MORNING'
```

This makes the assembler reserve the number of bytes of memory equal to the number of characters in the string named MESSAGE and initialise those locations by the ASCII equivalent of these characters.

```
VALUE    DB  50H
```

This statement directs the assembler to reserve 50H memory bytes and leave them uninitialised for the variable named VALUE.

DW: Define Word
The DW directive serves the same purposes as the DB directive, but it now makes
the assembler reserve the number of memory words (16-bit) instead of bytes. Some examples are given to explain this directive.



```
WORDS    DW  1234H, 4567H, 78ABH, 045CH,
```

This makes the assembler reserve four words in memory (8 bytes), and initialize the words with the specified values in the statements. During initialisation, the lower bytes are stored at the lower memory addresses, while the upper bytes are stored at the higher addresses. Another option of the DW directive is explained with the DUP operator.

```
WDATA    DW 5 DUP (6666H)
```

This statement reserves five words, i.e. 10-bytes of memory for a word lable WDATA and initialises all the word locations with 6666H.

DQ: Define Quadword
This directive is used to direct the assembler to reserve 4 words (8 bytes) of
memory for the specified variable and may initialise it with the specified values.
DT: Define Ten Bytes
The DT directive directs the assembler to define the specified variable requiring
10-bytes for its storage and initialise the 10-bytes with the specified values. The directive may be used in case
of variables facing heavy numerical calculations, generally processed by numerical processors.



ASSUME: Assume Logical Segment Name
The ASSUME directive is used to inform the as¬
semble, the names of the logicals segments to be assumed for different segments used in the program. In the assembly language program, each segment is given a name. For example, the code segment may be given the name CODE, data segment may be given the name DATA etc. The statement ASSUME CS : CODE directs the assembler that the machine codes are available in a segment named CODE, and hence the CS register is to be loaded with the address (segment) allotted by the operating system for the label CODE, while loading. Similary, ASSUME DS : DATA indicates to the assembler that the data items related to the program, are available in a logical segment named DATA, and the DS register is to be initialised by the segment address value decided by the operating system for the data segment, while loading. It then considers the segment DATA as a default data segment for each memory operation, related to the data and the segment CODE as a source segment for the machine codes of the program. The ASSUME statement is a must at the starting of each assembly language program, without which a message ‘CODE/DATA EMITTED WITHOUT SEG¬
MENT’ may be issued by an assembler.

END: END of Program
The END directive marks the end of an assembly language program. When
the assembler comes across this END directive, it ignores the source lines available later on. Hence, it should be ensured that the END statement should be the last statement in the file and should not appear in between. Also, no useful program statement should lie in the file, after the END statement.



ENDP: END of Procedure
In assembly language programming, the subroutines are called proce¬
dures. They may be independent program modules which return particular results or values to the calling programs. The ENDP directive is used to indicate the end of a procedure. A procedure is usually assigned a name, i.e. label. To mark the end of a particular procedure, the name of the procedure, i.e. label may appear
as a prefix with the directive ENDP. The statements, appearing in the same module but after the ENDP direc¬
tive, are neglected from that procedure. The structure given below explains the use of ENDP.

```
PROCEDURE STAR
      ⋮
   STAR ENDP
```



ENDS: END of Segment
This directive marks the end of a logical segment. The logical segments
are assigned with the names using the ASSUME directive. The names appear with the ENDS directive
as prefixes to mark the end of those particular segments. Whatever are the contents of the segments, they
should appear in the program before ENDS. Any statement appearing after ENDS will be neglected from the
segment. The structure shown below explains the fact more clearly.

```
DATA        SEGMENT
              ⋮
DATA        ENDS
ASSUME      CS : CODE, DS : DATA
CODE        SEGMENT
              ⋮
CODE        ENDS
END
```

The above structure represents a simple program containing two segments named DATA and CODE. The data related to the program must lie between the DATA SEGMENT and DATA ENDS statements. Similary, all the executable instructions must lie between CODE SEGMENT and CODE ENDS statements.



EVEN: Align on Even Memory Address
The assembler, while starting the assembling procedure of any program, initialises a location counter and goes on updating it, as the assembly proceeds. It goes on assigning the available addresses, i.e. the contents of the location counter, sequentially to the program vari¬ ables, constants and modules as per their requirements, in the sequence in which they appear in the program. The EVEN directive updates the location counter to the next even address, if the current location counter contents are not even, and assigns the following routine or variable or constant to that address. The structure given below explains the directive.

```
      EVEN
PROCEDURE     ROOT
                ⋮
      ROOT    ENDP
```

The above structure shows a procedure ROOT that is to be aligned at an even address. The assembler will start assembling the main program calling ROOT. When the assembler comes across the directive EVEN, it checks the contents of the location counter. If it is odd, it is updated to the next even value and then the ROOT procedure is assigned to that address, i.e. the updated contents of the location counter. If the content of the location counter is already even, then the ROOT procedure will be assigned with the same address.



EQU: Equate

- The directive EQU is used to assign a label with a value or a symbol. The use of this di¬ rective is just to reduce the recurrence of the numerical values or constants in a program code. The recurring value is assigned with a label, and that label is used in place of that numerical value, throughout the program.

While assembling, whenever the assembler comes across the label, it substitutes the numerical value for that label and finds out the equivalent code. Using the EQU directive, even an instruction mnemonic can be assigned with a label, which can then be used in the program in place of that mnemonic. Suppose, a numeri¬ cal constant which appears in a program ten times. If that constant is to be changed at a later time, one will have to make the correction 10 times. This may lead to human errors, because it is possible that a human programmer may miss one of those corrections. This will result in the generation of wrong codes. If the EQU directive is used to assign the value with a label that can be used in place of each recurrence of that constant, only one change in the EQU statement will give the correct and modified code. The examples given below show the syntax.

Example

```
LABEL      EQU    0500H
ADDITION   EQU    ADD
```

The first statement assigns the constant 500H with the label LABEL, while the second statement assigns another label ADDITION with mnemonic ADD.



EXTRN: External and PUBLIC: Public

- The directive EXTRN informs the assembler that the names, procedures and labels declared after this directive have already been defined in some other assembly language modules.
- While in the other module, where the names, procedures and labels actually appear, they must be declared public, using the PUBLIC directive. If one wants to call a procedure FACTORIAL appearing in MODULE1 from MODULE 2; in MODULE1, it must be declared PUBLIC using the statement PUBLIC FACTORIAL and in module 2, it must be declared external using the declaration EXTRN FACTORIAL.
- The statement of declaration EXTRN must be accompained by the SEGMENT and ENDS directives of the MODULE 1, before it is called in MOBULE 2.
- Thus the MODULE1 and MODULE 2 must have the following declarations.

```
MODULE1     SEGMENT
PUBLIC      FACTORIAL FAR
MODULE1     ENDS
MODULE2     SEGMENT
EXTRN       FACTORIAL FAR
MODULE2     ENDS
```



GROUP: Group the Related Segments
This directive is used to form logical groups of segments
with similar purpose or type. This directive is used to inform the assembler to form a logical group of the fol¬
lowing segment names. The assembler passes an information to the linker/loader to form the code such that
the group declared segments or operands must lie within a 64Kbyte memory segment. Thus all such segments
and labels can be addressed using the same segment base.

```
PROGRAM
GROUP
CODE ,
DATA ,
STACK
```

The above statement directs the loader/linker to prepare an EXE file such that CODE, DATA and STACK
segment must lie within a 64kbyte memory segment that is named as PROGRAM. Now, for the ASSUME
statement, one can use the label PROGRAM rather than CODE, DATA and STACK as shown.

```
ASSUME
CS:
PROGRAM ,
```

LABEL: Label
The Label directive is used toassigna name to the current content of the location counter.
When the assembly process starts, the assembler initialises a location counter to keep track of memory loca¬
tions assigned to the program. As the program assembly proceeds, the contents of the location counter are
updated. During the assembly process, whenever the assembler comes across the LABEL directive, it assigns
the declared label with the current contents of the location counter. The type of the label must be specified, i.e.
whether it is a NEAR or a FAR label, BYTE or WORD label, etc.
A LABEL directive may be used to make a FAR jump as shown below. A FAR jump cannot be made at a normal label with a colon. The label CONTINUE can be used for a FAR jump, if the program contains the
following statement.
CONTINUE
LABEL
FAR
The LABEL directive can be used to refer to the data segment along with the data type, byte or word as
shown.



```
DATA        SEGMENT
DATAS DB 50H DUP (?)
DATA-LAST LABEL BYTE FAR
DATA ENDS
```

After reserving 50H locations for DATAS, the next location will be assigned a label DATA-LAST and its type will be byte and far.

**LENGTH: Byte Length of a Label** This directive is not available in MASM. This is used to refer to the length of a data array or a string.

```
MOV CX, LENGTH ARRAY
```

This statement, when assembled, will substitute the length of the array ARRAY in bytes, in the instruction.

**LOCAL** The lables, variables, constants or procedures declared LOCAL in a module are to be used only by that particular module. After some time, some other module may declare a particular data type LOCAL, which was previously declared LOCAL by an other module or modules. Thus the same label may serve different purposes for different modules of a program. With a single declaration statement, a number of variables can be declared local, as shown.

```
LOCAL a, b, DATA, ARRAY, ROUTINE
```



**NAME: Logical Name of a Module** The NAME directive is used to assign a name to an assembly language program module. The module, may now be referred to by its declared name. The names, if selected to be suggestive, may point out the functions of the different modules and hence may help in the documentation.

**OFFSET: Offset of a Label** When the assembler comes across the OFFSET operator along with a label, it first computes the 16-bit displacement (also called as offset interchangebly) of the particular label, and replaces the string 'OFFSET LABEL' by the computed displacement. This operator is used with arrays, strings, lables and procedures to decide their offsets in their default segments. The segment may also be decided by another operator of similar type, viz, SEG. Its most common use is in the case of the indirect, indexed, based indexed or other addressing techniques of similar types, used to refer to the memory indirectly. The examples of this operator are as follows:

Example

```
CODE SEGMENT
MOV SI, OFFSET LIST
CODE ENDS
DATA SEGMENT
LIST DB 10H
DATA ENDS
```



**ORG : Origin** The ORG directive directs the assembler to start the memory allotment for the particular segment, block or code from the declared address in the ORG statement. While starting the assembly process for a module, the assembler initialises a location counter to keep track of the allotted addresses for the module. If the ORG statement is not written in the program, the location counter is initialised to 0000. If an ORG 200H statement is present at the starting of the code segment of that module, then the code will start from 200H address in code segment. In other words, the location counter will get initialised to the address 0200H instead of 0000H. Thus, the code for different modules and segments can be located in the available memory as required by the programmer. The ORG directive can even be used with data segments similarly.

**PROC: Procedure** The PROC directive marks the start of a named procedure in the statement. Also, the types NEAR or FAR specify the type of the procedure, i.e whether it is to be called by the main program located within 64K of physical memory or not. For example, the statement RESULT PROC NEAR marks the start of a routine RESULT, which is to be called by a program located in the same segment of memory. The FAR directive is used for the procedures to be called by the programs located in different segments of memory. The example statements are as follows:

```
RESULT    PROC   NEAR
ROUTINE   PROC   FAR
```



PTR: Pointer

- The POINTER operator is used to declare the type of a label, variable or memory operand.
- The operator PTR is prefixed by either BYTE or WORD. If the prefix is BYTE, then the particular label, variable or memory operand is treated as an 8-bit quantity, while if WORD is the prefix, then it is treated as a 16-bit quantity.
- In other words, the PTR operator is used to specify the data type—byte or word.

- The examples of the PTR operator are as follows:

```
MOV AL, BYTE PTR [SI] -       Moves content of memory location
                              addressed by SI (8-bit) to AL
INC BYTE PTR [BX]-            Increments byte contents of memory
                              location addressed by BX
MOV BX, WORD PTR [2000H]-     Moves 16-bit content of memory
                              location 2000H to BX, i.e. [2000H] to BL
                              [2001H] to BH
INC WORD PTR [3000H] -        Increments word contents of memory
                              location 3000H considering contents
                              of 3000H (lower byte) and 3001H
                              (higher byte) as a 16-bit number
```

In case of JMP instructions, the PTR operator is used to specify the type of the jump, i.e. near or far, as explained in the examples given below.

JMP
NEAR
PTR [ BX ] - NEAR Jump
JMP
FAR
PTR [ BX ]- FAR Jump.



PUBLIC

- the PUBLIC directive is used along with the EXTRN directive.
- This informs the assembler that the labels, variables, constants, or procedures declared PUBLIC may be accessed by other assembly modules to form their codes, but while using the PUBLIC declared lables, variables, constants or procedures the user must declare them externals using the EXTRN directive.
- On the other hand, the data types declared EXTRN in a module of the program, may be declared PUBLIC in at least any one of the other modules of the same program.
- (Refer to the explanation on EXTRN directive to get the clear idea of PUBLIC.)

SEG: Segment of a Label

- The SEG operator is used to decide the segment address of the label, variable, or procedure and substitutes the segment base address in place of “SEG” label.
- The example given below explains the use of SEG operator.

```
MOV AX, SEG ARRAY  ; This statement moves the segment address of ARRAY in
MOV DS, AX         ; which it is appearing, to register AX and then to DS.
```



**SEGMENT: Logical Segment** The SEGMENT directive marks the starting of a logical segment. The started segment is also assigned a name, i.e. label, by this statement. The SEGMENT and ENDS directive must bracket each logical segment of a program. In some cases, the segment may be assigned a type like PUBLIC (i.e. can be used by other modules of the program while linking) or GLOBAL (can be accessed by any other modules). The program structure given below explains the use of the SEGMENT directive.

```
EXE.CODE SEGMENT GLOBAL; Start of Segment named EXE.CODE,
                       ; that can be accessed by any other module.
EXE.CODE ENDS          ; END of EXE.CODE logical segment.
```

**SHORT** The SHORT operator indicates to the assembler that only one byte is required to code the displacement for a jump (i.e. displacement is within –128 to +127 bytes from the address of the byte next to the jump opcode). This method of specifying the jump address saves the memory. Otherwise, the assembler may reserve two bytes for the displacement. The syntax of the statement is as given below.

```
JMP SHORT LABEL
```

**TYPE** The TYPE operator directs the assembler to decide the data type of the specified label and replaces the 'TYPE' label by the decided data type. For the word type variable, the data type is 2, for double word type, it is 4, and for byte type, it is 1. Suppose, the STRING is a word array. The instruction MOV AX, TYPE STRING moves the value 0002H in AX.

**GLOBAL** The labels, variables, constants or procedures declared GLOBAL may be used by other modules of the program. Once a variable is declared GLOBAL, it can be used by any module in the program. The following statement declares the procedure ROUTINE as a global label.

```
ROUTINE PROC     GLOBAL
```

**'+ & –' Operators** These operators represent arithmetic addition and subtraction respectively and are typically used to add or subtract displacements (8 or 16 bit) to base or index registers or stack or base pointers as given in the example:



```
MOV AL, [ SI +2 ]
MOV DX, [ BX – 5 ]
MOV BX, [ OFFSET LABEL + 10 H ]
MOV AX, [ BX + 9I ]
```

**FAR PTR** This directive indicates the assembler that the label following FAR PTR is not available within the same segment and the address of the label is of 32-bits i.e. 2 bytes offset followed by 2 bytes segment address.

```
JMP FAR PTR LABEL
CALL FAR PTR ROUTINE
```

- Both the above instructions indicate to the assembles that the target address is going to require four bytes;
- Lower byte of offset, higher byte of offset, lower byte of segment and higher byte of segment; indicating intersegment addressing mode.



NEAR PTR
This directive indicates that the label following NEAR PTR is in the same segment and needs only 16 bit i.e. 2 byte offset to address it.

Example

```
JMP NEAR PTR LABEL
CALL NEAR PTR ROUTINE
```

- If a label is not preceded by NEAR PTR or FAR PTR, then it is by default considered a NEAR PTR label and two bytes are reserved by the assembler for its address during the process of assembling.

Dos and Don'ts While Using Instructions

1) The logic required for implementing a program must be visualized very clearly. There may be many
methods or alternative logics to implement a program. A simple-to-implement method must be selected.

2) Express the selected logic in terms of a flowchart or algorithm.

3) Identify the most appropriate instructions to implement the algorithmic steps of the logic.

4) Arrange them in logical sequence to solve the problem.



5) Use more efficient instructions in terms of length in bytes and execution time for implementing the logic. For example, one can use INC AL instead of ADD AL, 01H.

6) Simulate the program on paper assuming a few possible sets of inputs.

7) Provide comments with each instruction regarding its role in the overall implementation of the logic.

8) Remove unnecessary or redundant instructions from the program.

9) If the simulation in 6) goes wrong, repeat from Step 3.

**Don'ts While Using Instructions**

1) Don't use any instruction, till its operation is very clear to you. You must be convinced about its role in implementing the logic.
2) Don't use unnecessarily complex addressing modes and instructions, until there is no simpler alternative. Indirect addressing modes must be used conservatively.
3) Don't use mismatching size operands in any instruction until it is demanded by the operation.

```
MOV AX, DL          ; Not allowed.
DIV BL              ; Divide DX-AX (32 bit)
                    ; by BL (8 bit) is allowed.
```

4) Don't use both the operands in an instruction as memory operands. Only one memory operand can be specified in one instruction.

```
MOV [SI], [DI]      ; Not allowed.
MOV AX, [SI]              ; Allowed.
```



5) Don't use immediate 8-bit or 16-bit operand as a destination operand. Destination operand must be a storage element like a register or a memory location. The immediate operand can only be a source operand.

```
MOV 55H, AL         ; Not allowed.
ADD 5779H, AX       ; Not allowed.
ADD AX, 5779H       ; Allowed.
```

6) Both the operands of an instruction can't be immediate operands.
7) Don't use stack operations unnecessarily. Stack operations must be used very carefully.
8) Don't write very big continuous single program for an application. Rather divide the big task in small modules and write modular programmes using subroutines or interrupt service routines if required.
9) Don't use segment registers as operands for arithmatic or logical instructions. It is not allowed.